

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

Работа допущена к защите
Директор ВШПИ
_____ П. Д. Дробинцев
«___» _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
работа бакалавра

**ИНТЕГРИРОВАННЫЕ СРЕДСТВА ПОМОЩИ В
ПУТЕШЕСТВИЯХ**

по направлению подготовки
**02.03.02 Фундаментальная информатика и информационные техноло-
гии**

Направленность (профиль)

02.03.02_02 Информатика и компьютерные науки

Выполнил студент гр. _____ С. В. Рушкевич
5130202/20202

Руководитель, старший преподаватель _____ Самочадина Т. Н.

Консультант по нормоконтролю _____ Самочадина Т. Н.

Санкт-Петербург
2026

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО

Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

УТВЕРЖДАЮ

Директор ВШПИ

_____ П. Д. Дробинцев

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студенту Рушкевичу Станиславу Владимировичу, группа 5130202/20202

1. Тема работы: Интегрированные средства помощи в путешествиях

2. Срок сдачи студентом законченной работы: 01.06.2026

3. Исходные данные по работе: требования к мобильному приложению для совместного планирования путешествий; документация программных интерфейсов сервисов Yandex MapKit, Geoapify и OpenRouteService; техническая документация платформы Android и библиотек Android Jetpack; научные публикации по алгоритмам построения и оптимизации маршрутов.

4. Содержание работы (перечень подлежащих разработке вопросов):

- Актуальность разработки;
- анализ существующих решений для планирования путешествий;
- проектирование архитектуры мобильного приложения;
- разработка алгоритмов оптимизации маршрута и офлайн-синхронизации данных;
- тестирование приложения и анализ полученных результатов.

5. Перечень графического материала (с указанием обязательных чертежей): нет.

6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):

- язык программирования: Kotlin;
- фреймворки: Android SDK, Jetpack Compose;
- библиотеки: Hilt, Room, DataStore, Retrofit, OkHttp, Navigation Compose, Kotlin Coroutines, Yandex MapKit, Google ML Kit;

- облачные сервисы: Firebase, Geoapify, OpenRouteService, Yandex Geosuggest;
- инструмент сборки: Gradle;
- сквозные технологии: Git, GitHub, Android Studio.

7. Консультанты по работе: нет.

8. Дата выдачи задания: 07.04.2026

Руководитель ВКР _____ (подпись) Самочадина Т. Н.

Задание принял к исполнению 07.04.2026

Студент _____ (подпись) С. В. Рушкевич

РЕФЕРАТ

На 65 с., 12 рисунков, 4 таблицы, 2 приложения.

КЛЮЧЕВЫЕ СЛОВА: МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, ANDROID, JETPACK COMPOSE, OFFLINE-FIRST, BLUETOOTH CLASSIC, AES-256, ПЛАНИРОВАНИЕ ПУТЕШЕСТВИЙ, ОПТИМИЗАЦИЯ МАРШРУТА, МУЛЬТИВАЛЮТНЫЙ УЧЁТ РАСХОДОВ.

Тема выпускной квалификационной работы: «Интегрированные средства помощи в путешествиях».

Предмет работы — программные средства и методы построения мобильного помощника путешественника, в котором планирование по дням, поиск мест и навигация, мультивалютный учёт расходов и групповая координация собраны в одном приложении, и в котором ключевые сценарии работают без подключения к интернету.

Цель работы — разработать мобильное приложение Triloo для платформы Android, дающий пользователю единую среду подготовки и сопровождения поездки и позволяющий обмениваться структурированными данными поездки между устройствами без сети.

Методы работы: системный анализ предметной области, сравнительный анализ существующих решений, проектирование многоуровневой offline-first-архитектуры по шаблону MVVM, эвристика ближайшего соседа поверх реальных дистанций из маршрутных API, симметричное шифрование AES-256 в режиме GCM, юнит-тестирование на JVM.

Основные результаты: спроектирована и реализована архитектура мобильного клиента с локальным хранилищем Room и реактивными потоками Kotlin Flow; реализовано приложение со связкой «план — карта — расходы — группа»; разработана подсистема Triloo Relay для офлайн-обмена пакетами данных поездки по Bluetooth Classic; подготовлена онлайн-синхронизация с собственным backend.

Область применения — мобильные приложения для подготовки и сопровождения групповых путешествий, а также как референсная архитектура офлайн-обмена данными между устройствами без подключения к интернету.

ABSTRACT

65 pages, 12 figures, 4 tables, 2 appendices.

KEYWORDS: MOBILE APPLICATION, ANDROID, JETPACK COMPOSE, OFFLINE-FIRST, BLUETOOTH CLASSIC, AES-256, TRIP PLANNING, ROUTE OPTIMIZATION, MULTI-CURRENCY EXPENSE TRACKING.

The subject of the graduate qualification work is «Integrated tools for assistance in travel».

The work focuses on software methods and components for a mobile travel companion that combines day-by-day itinerary planning, place search and navigation, multi-currency expense tracking and group coordination in a single application, with key scenarios available without an internet connection.

The goal of the work is to develop the Triloo mobile application for the Android platform that provides a single environment for trip preparation and accompaniment and that allows participants to exchange structured trip data between devices with no network in between.

Methods used: systems analysis of the problem domain, comparative analysis of existing solutions, design of a layered offline-first architecture following the MVVM pattern, nearest-neighbour heuristic on top of real distances obtained from routing APIs, symmetric AES-256 encryption in GCM mode, unit testing on the JVM.

Main results: an architecture of a mobile client based on a local Room database and reactive Kotlin Flow streams has been designed and implemented; an application that integrates the «plan — map — expenses — group» loop has been built; the Triloo Relay subsystem for offline exchange of trip data packages over Bluetooth Classic has been developed; an online synchronisation channel with a custom backend over the same domain layer has been prepared.

Application area — mobile applications for the preparation and accompaniment of group trips, as well as a reference architecture for offline exchange of structured data between devices without an internet connection.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
ГЛАВА 1. ЗАДАЧА ЕДИНОГО МОБИЛЬНОГО ПОМОЩНИКА ДЛЯ ПОДГОТОВКИ И СОПРОВОЖДЕНИЯ ГРУППОВЫХ ПОЕЗДОВ	12
1.1. Характеристика задачи интегрированного помощника путешественника	12
1.2. Современное состояние области: обзор существующих приложений и подходов	15
1.3. Объект и методы разработки	21
1.4. Уточнённые требования к разрабатываемому приложению Triloo	23
ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ TRILOO	29
2.1. Многоуровневая offline-first-архитектура клиента	29
2.2. Доменная модель поездки и локальное хранилище	32
2.3. Интеграционный слой со сторонними сервисами	34
2.4. Подсистема офлайн-обмена Triloo Relay	36
2.5. Выводы по главе 2	39
ГЛАВА 3. РЕАЛИЗАЦИЯ КЛЮЧЕВЫХ ПОДСИСТЕМ TRILOO	40
3.1. Эвристическая оптимизация порядка обхода точек	40
3.2. Мультивалютный расчёт балансов и упрощение долгов	42
3.3. Протокол обмена Triloo Relay	44
3.4. Онлайн-синхронизация с собственным backend	45
3.5. Выводы по главе 3	46
ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА ПРИЛОЖЕНИЯ TRILOO	47
4.1. Методика и среда проверки	47
4.2. Покрывание функциональных требований	47
4.3. Качество маршрута на выходе оптимизатора	50
4.4. Корректность слияния пакетов Triloo Relay	50
4.5. Безопасность канала Triloo Relay	51
4.6. Поведение в офлайн-режиме	51
4.7. Выводы по главе 4	52

ВВЕДЕНИЕ

Подготовка поездки и поведение в самой поездке — это работа сразу с несколькими разнородными задачами: расписание по дням, поиск мест и навигация, бюджет с разными валютами, согласование решений в группе участников. В массовых пользовательских сценариях каждая из этих задач обычно закрывается отдельным приложением: карта или картографический сервис, заметка или таблица, групповой мессенджер, узкоспециализированный сервис для деления расходов. Адрес встречи в результате лежит в карте, время — в чате, кто кому сколько должен — в третьем сервисе. У разных участников быстро расходятся локальные версии плана; данные приходится вручную переносить между приложениями. Поверх всего этого ложится нестабильная сеть: значительная часть поездки проходит в роуминге, без Wi-Fi или с очень медленным мобильным интернетом, и облачные сервисы в таких условиях работают плохо или не работают совсем.

В типовом сценарии подготовки путешествия используется четыре–шесть разных сервисов, и ни один из них не закрывает связку «план — карта — расходы — группа» целиком. Доступные интегрированные планировщики (Wanderlog, Roadtrippers, Sygic Travel, TripFlow, FlowTrip) берут на себя часть сценария, но в офлайн-режиме либо переходят к просмотру ранее загруженных данных, либо отказывают совсем. Картографические продукты (Yandex Maps, Google Maps, 2GIS) уверенно решают задачи поиска места и навигации, но не предоставляют структурированного плана и совместного редактирования. Финансовые приложения класса Splitwise умеют считать долги в группе, но изолированы от плана и маршрута поездки. Способы обмениваться структурированными данными поездки между устройствами без интернета — например, в самолёте, в горах или в стране без покрытия мобильной сети — в массовых продуктах сейчас фактически нет: остаются файловый импорт-экспорт и мессенджеры, которые не сводят версии и не пересчитывают связанные сущности.

Актуальность работы определяется растущим числом путешественников, для которых типична связка «планирую заранее, в поездке использую план в нестабильной сети, делю расходы в группе из 2–5 человек». Эту аудиторию обслуживают перечисленные выше классы продуктов, но в типовом сценарии группа всё равно сводит данные руками, и попытки согласовать план без интернета упираются в архитектурные ограничения облачных сервисов. Заметная доля задач, связанных с поездкой, локальна по своей природе: дни, места, расходы, участники не требуют ни глобального состояния, ни постоянной онлайн-консистентности. Это даёт пространство для прикладной разработки — мобильного приложения, в котором перечисленные функции собраны вместе и в котором есть механизм согласовать данные между устройствами без интернета.

Степень разработанности проблемы. Сами подходы, на которые опирается такая задача, развиты по отдельности. Эвристики решения задачи коммиво-

яжёра — в частности, алгоритм ближайшего соседа и его расширения с фиксированными концами — давно применяются в логистике и доступны через маршрутные API (OpenRouteService, Yandex Transport). Паттерны offline-first для мобильных и веб-клиентов разобраны в литературе по PWA и мобильной разработке. Симметричное шифрование AES в режиме GCM описано в спецификациях NIST и стандартно используется в защищённых каналах. Bluetooth Classic с профилем RFCOMM применяется для P2P-обмена между устройствами и описан в спецификации Bluetooth SIG. В массовых тревел-продуктах эти подходы вместе не собирают: продуктовая ниша «интегрированный помощник путешественника с офлайн-обменом данными в группе» остаётся незанятой, и обзор существующих решений в первой главе работы это подтверждает.

Объект исследования — процесс подготовки и сопровождения групповых путешествий с использованием мобильных программных средств. **Предметом исследования** выступают программные средства и методы интегрированного помощника путешественника, обеспечивающие единую среду планирования по дням, поиска мест, мультивалютного учёта расходов и групповой координации, а также офлайн-обмен структурированными данными поездки между устройствами.

Цель работы — разработать мобильное приложение Triloo для платформы Android, дающий пользователю единую среду подготовки и сопровождения поездки и позволяющий обмениваться структурированными данными поездки между устройствами без интернета.

Для достижения цели в работе решаются следующие **задачи**:

1. Проанализировать сценарии подготовки и сопровождения групповых поездок, провести обзор существующих приложений и подходов и сформулировать обобщённую постановку задачи, функциональные и нефункциональные требования к интегрированному мобильному помощнику.
2. Спроектировать многоуровневую offline-first-архитектуру мобильного клиента, доменную модель поездки и принципы работы с внешними сервисами так, чтобы ключевые сценарии работали без сети.
3. Реализовать приложение Triloo для Android с поддержкой планирования по дням, поиска мест и работы с картой, мультивалютного учёта расходов с расчётом сплитов и долгов, групповых сценариев с приглашениями и геотаргетингом, а также распознавания чеков и рекомендаций жилья.
4. Разработать и реализовать подсистему офлайн-обмена данными поездки Triloo Relay поверх Bluetooth Classic с симметричным шифрованием AES-256-GCM, а также подготовить онлайн-синхронизацию с собственным backend поверх единого доменного слоя.
5. Провести экспериментальную проверку приложения: оценить корректность слияния пакетов при офлайн-синхронизации, качество маршрута на выходе оптимизатора и поведение приложения в типовых офлайн-сценариях.

Теоретическую и методологическую базу работы составили: классические работы по шаблонам проектирования и архитектуре программных систем; материалы по разработке под Android с использованием Jetpack Compose, Room, Hilt и Kotlin Coroutines; публикации по offline-first-архитектурам мобильных приложений; работы по эвристикам решения задачи коммивояжёра; стандарты NIST по AES-GCM и спецификации Bluetooth SIG. **Методы исследования:** системный анализ предметной области и сравнительный анализ существующих решений на стадии постановки задачи; объектно-ориентированное проектирование и шаблон MVVM на стадии проектирования архитектуры; разработка приложения на Kotlin с использованием Jetpack Compose и Room; юнит-тестирование на JVM (JUnit, Robolectric) для верификации доменной логики (расчёт балансов, оптимизация маршрута, слияние пакетов Relay).

Элементы новизны работы носят прикладной характер и состоят в следующем: предложена интеграция офлайн-обмена структурированными данными поездки через Bluetooth Classic с типовой моделью «поездка — день — место — расход — участник» в одном мобильном приложении; журнал удалений (DeletionLog) вынесен в отдельную сущность доменной модели и используется при слиянии пакетов как самостоятельный артефакт, что предотвращает «воскрешение» удалённых записей при последующих синхронизациях; реализован единый доменный слой для офлайн-обмена через Bluetooth и онлайн-синхронизации с backend — транспорт меняется без изменений в репозиториях и UI.

Практическая значимость работы заключается в следующем: построено мобильное приложение для подготовки и сопровождения групповых поездок, в котором связка «план — карта — расходы — группа» собрана в одном продукте; реализована подсистема Triloo Relay, которая закрывает разрыв в офлайн-обмене данными поездки между устройствами и применима как референсная архитектура в близких задачах; код приложения структурирован так, что подсистему синхронизации можно использовать поверх других транспортов (Wi-Fi Direct, mesh-сети) без изменений в доменном слое; результаты обзора существующих решений в первой главе пригодны для уточнения продуктовой постановки задач в смежных тревел-продуктах.

Структура работы. Работа состоит из введения, четырёх глав, заключения, списка использованных источников и приложений. В **первой главе** даётся характеристика решаемой задачи, проводится обзор существующих приложений и подходов, определяются объект и методы разработки и формулируются уточнённые требования к разрабатываемому приложению. **Вторая глава** описывает архитектуру приложения: многоуровневую offline-first-схему клиента, доменную модель и локальное хранилище Room, интеграционный слой со сторонними сервисами и подсистему офлайн-обмена Triloo Relay. **Третья глава** посвящена реализации ключевых подсистем: эвристической оптимизации маршрута, мультивалютному расчёту балансов и упрощению долгов, протоколу обмена в Triloo Relay и онлайн-синхронизации с собственным backend. В **четвёртой**

главе приведены результаты экспериментальной проверки приложения: корректность слияния пакетов Triloo Relay, качество маршрута на выходе оптимизатора, поведение приложения в офлайн-сценариях. В **заключении** подведены итоги, сформулированы предложения по дальнейшему развитию работы, в частности переход от Last-Write-Wins к слиянию на уровне отдельных полей и поддержка дополнительных транспортов офлайн-обмена.

ГЛАВА 1. ЗАДАЧА ЕДИНОГО МОБИЛЬНОГО ПОМОЩНИКА ДЛЯ ПОДГОТОВКИ И СОПРОВОЖДЕНИЯ ГРУППОВЫХ ПОЕЗДОК

1.1. Характеристика задачи интегрированного помощника путешественника

Поездка с точки зрения её организатора складывается не из одной задачи, а из нескольких слабо связанных работ, которые приходится вести одновременно. Расписание раскладывает дни поездки и распределяет по ним достопримечательности, рестораны, переезды и свободные блоки. Навигация привязывает места к координатам и считает, сколько уйдёт на дорогу. Бюджет фиксирует траты в той валюте, в которой они идут, и переводит итог в домашнюю. Координация в группе сводит решения двух–пяти участников к одному состоянию. Поверх этих слоёв ложится сеть, которая в реальной поездке нестабильна по объективным причинам: роуминг, отсутствие Wi-Fi, движение по маршруту.

Массовая практика закрывает каждый слой отдельным приложением. Расписание ведётся в заметках или таблицах (*Notion* [22], *Obsidian* [23], *Google Sheets*, системные заметки). Навигация лежит в картах: *Yandex Maps*, *Google Maps*, *2GIS*. Группа сидит в мессенджере. Расходы попадают в *Splitwise* [27], *Tricount* [28] или *Settle Up* [29]. Связки между этими инструментами нет, и одно и то же кафе живёт сразу в нескольких сервисах: адрес в заметке, координаты в закладках карты, оплата счёта в *Splitwise*, время встречи в чате. Изменения в одном слое не транслируются в другие, и через пару дней у каждого участника на руках своя версия плана. Типовой состав приложений на поездку у российского пользователя — четыре–шесть продуктов, и ни одно из них не закрывает связку «план + карта + расходы + группа» целиком.

Из этой структуры вырастает несколько отдельных проблем, и именно они определяют постановку задачи.

Первая проблема — фрагментация данных. Один и тот же объект (место, событие, расход) одновременно существует в нескольких сервисах, и при любом изменении его приходится править вручную в каждом. Расход за обед в *Splitwise* не порождает отметки в плане дня. Закладка в *Yandex Maps* не привязывается к расписанию поездки. Обсуждения в чате не превращаются в решения, видимые всей группой. Чтобы снять фрагментацию, нужна единая доменная модель, в которой «поездка, день, место, расход, участник» связаны явными ссылками и принадлежат одному хранилищу.

Вторая проблема — зависимость от сети. Большинство интегрированных планировщиков (*Wanderlog*, *Roadtrippers*, *Sygyt Travel* в бесплатной версии, *TripFlow*, *FlowTrip*) технически являются тонкими клиентами к облачному API. Интерфейс работает поверх постоянного запроса к серверу, локальный кэш доступен только в режиме просмотра. На городском Wi-Fi такое поведение

терпимо, но плохо переносит маршрут: в самолёте, в поезде, в горах, в стране без покрытия сотовой сети у пользователя на руках остаётся либо устаревший снимок плана, либо крутящееся колесо загрузки. При этом значительная часть задач путешествия локальна по существу и не требует сетевого взаимодействия: посмотреть план на день, добавить расход к месту, отметить точку как посещённую, пересчитать долги [1, 2].

Третья проблема — отсутствие способа обмениваться структурированными данными между устройствами без интернета. Когда два участника группы оказываются рядом без сети, у них нет инструмента, который позволил бы передать с одного устройства на другое не файл и не сообщение, а пакет данных поездки (план, места, расходы, журнал изменений) с последующим корректным слиянием на принимающей стороне. Файловый импорт-экспорт через мессенджеры не сводит версии и не пересчитывает связанные сущности: балансы после добавления расхода, дублирующиеся места, изменения времени активности. В массовых тревел-продуктах такая функция отсутствует как класс.

Четвёртая проблема касается мультивалютности и группового расчёта долгов. Часть трат в поездке совершается в местной валюте, часть в домашней — например, бронирования, оплаченные заранее. Финансовому модулю приходится хранить и сумму в валюте операции, и эквивалент в базовой валюте поездки, и курс на момент конвертации. На массиве из десятков расходов в группе из нескольких человек ручной расчёт «кто кому сколько» становится трудоёмким и склонным к ошибкам. По существу задача сводится к тому, чтобы по списку расходов с распределением долей построить минимальный набор переводов, закрывающий все долги.

Пятая проблема — согласование решений в группе. Часть решений в групповой поездке принимается совместно: в какой день идти в музей, где встретиться в 19:00, кто бронирует следующий ночлег. Без явных ролей и общего состояния такие решения растворяются в чате. Минимальная модель группы включает список участников с ролями (OWNER, ADMIN, MEMBER), общий план поездки и опционально взаимный геошаринг.

Если сопоставить разрабатываемую задачу с близкими и смежными областями, она оказывается на пересечении нескольких прикладных направлений: проектирование offline-first-приложений [1, 3], эвристическая оптимизация порядка обхода точек, родственная задаче коммивояжёра с временными окнами [4, 5], криптография симметричного шифрования применительно к беспроводному каналу [6, 7], пиринговый обмен данными по Bluetooth [8], мультивалютный учёт расходов с курсами в роли справочной информации, групповая координация в небольших коллективах. Сами подходы и алгоритмы по этим направлениям давно развиты по отдельности. Эвристика ближайшего соседа описана ещё в 1977 году в работе Розенкранца, Стирнса и Льюиса [4]. AES в режиме GCM закреплён в стандарте NIST SP 800-38D [6]. Offline-first-паттерны разобраны в литературе по PWA и мобильной разработке [3]. В массовых тревел-продуктах их вместе не собирают, и каждая из задач остаётся в своей нише.

Значимость задачи раскрывается через конкретные группы пользователей. Одиночному путешественнику решение даёт меньше приложений в поездке и убирает ручной перенос данных между ними. Группе из 2–5 человек обеспечивает единое актуальное состояние плана и бюджета, которое не разваливается без сети. Российской аудитории даёт приложение, в котором карта, поиск мест и автодополнение собраны вокруг русскоязычных сервисов (Yandex MapKit [9], Yandex Geosuggest), а интерфейс не зависит от доступности сервисов Google в стране пребывания. Для области программной инженерии прикладной интерес работы состоит в демонстрации многоуровневой offline-first-архитектуры на современной Android-платформе (Jetpack Compose, Room, Kotlin Coroutines), в которой одна и та же доменная модель используется и для локальной работы, и для офлайн-обмена через Bluetooth, и для онлайн-синхронизации.

Целевой процесс подготовки и сопровождения поездки, который должно поддерживать разрабатываемое приложение, показан на рис. 1. Планирование по дням и ведение расходов дополнены ветвлением по наличию сети: когда участники оказываются рядом без интернета, данные передаются прямым офлайн-обменом между устройствами, иначе — синхронизируются через сеть.

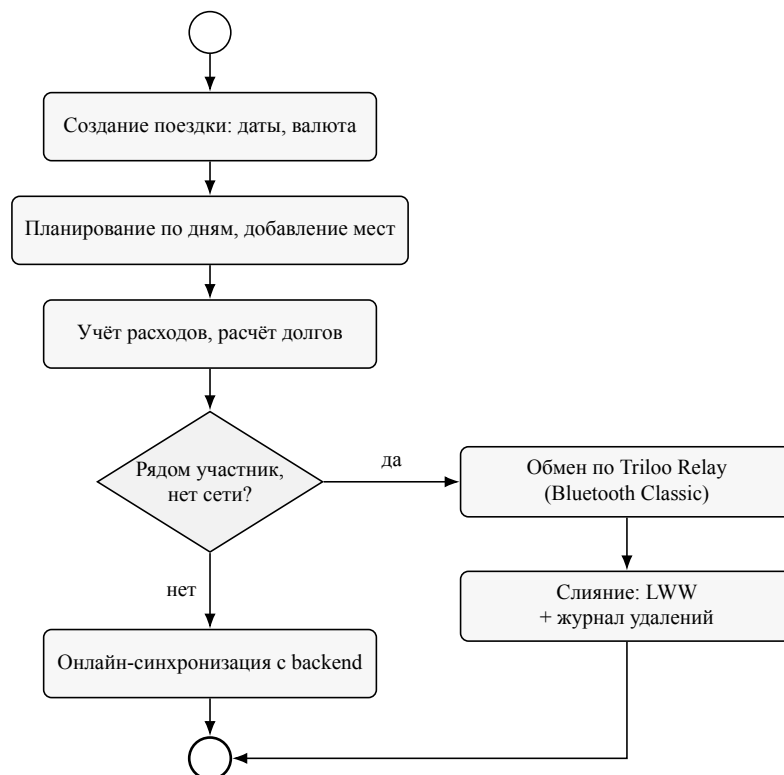


Рис. 1. BPMN-схема процесса подготовки и сопровождения поездки

Обобщённо задача формулируется так: нужно мобильное приложение для Android, в котором собраны планирование по дням, поиск мест и работа с картой, мультивалютный учёт расходов с расчётом долгов и групповые сценарии, и в которое встроен механизм обмена структурированными данными поездки между устройствами без интернета. Конкретные цель, задачи и требования

сформулированы в §1.4 после обзора существующих решений (§1.2) и определения объекта и методов разработки (§1.3).

1.2. Современное состояние области: обзор существующих приложений и подходов

Целевой сценарий «поездка как проект с планом, картой, расходами и группой, работающая в нестабильной сети» не закрывается ни одним классом существующих продуктов. По функциональному назначению на рынке выделяется девять групп массовых решений, между которыми пользователь обычно делит свой сценарий: универсальные инструменты ведения данных, итинерари-планировщики, тревелогги, готовые шаблоны маршрутов, агрегаторы бронирований, B2B-инструменты для турагентов, картографические приложения, финансовые приложения для разделения расходов, AI-планировщики. Ниже разобран каждый класс с указанием того, что в нём закрыто из целевого сценария и где проходят его границы. По ходу обзора фиксируется терминология, используемая в дальнейшем: поездка, день, активность, сплит, баланс, оптимизация маршрута.

Универсальные инструменты: заметки, таблицы, мессенджеры

Самый распространённый ручной подход — связка из заметки в *Notion* [22], *Obsidian* [23] или системных заметках, таблицы (*Google Sheets*) и группового чата. Плюсы такого решения — свободная структура данных и знакомый интерфейс. Минусы прямо отражают исходную проблему: единой модели «поездка, дни, места, расходы» в заметке нет, привязки к координатам и маршруту нет, согласование изменений идёт через чат в свободной форме, и у участников быстро расходятся локальные версии. Это не альтернатива специализированному инструменту, а его отсутствие, скомпенсированное усилием пользователя.

Itinerary-планировщики: Wanderlog, Roadtrippers, Sygic Travel

Класс приложений, целевая модель которых — последовательность дней с привязанными активностями. *Wanderlog* [18] позволяет создать поездку, добавить даты, искать места по *Google Places*, распределять их по дням, видеть всё на карте и делиться ссылкой на план. *Roadtrippers* [19] ориентирован на автомобильные путешествия и считает между точками расстояние, время в пути и оценочные расходы на бензин. *Sygic Travel* [20] строит планы поверх собственной POI-базы и в премиум-версии умеет скачивать карты целых городов для офлайн-просмотра.

Ограничения у всех трёх похожи. Финансовый контур или отсутствует, или сделан поверхностно (плоский список расходов без сплитов и валют). Совместное редактирование возможно только при подключении к серверу. Оптимиза-

ция порядка точек, если она есть, эвристическая и не учитывает реальное время в пути и часы работы заведений. Базовая модель «поездка, день, место» у этого класса достаточно удобная, и разрабатываемое приложение её перенимает, добавляя финансовый контур и офлайн-обмен данными.

Тревелогги: Polarsteps

Polarsteps [24] — приложение для ведения дневника поездки: фиксация маршрута через GPS-трекер, статистика пройденных километров, фотокарта, общедоступная страница путешествия. Это смежный, но другой сценарий: тревелог фиксирует прошедшую поездку, а не планирует предстоящую. Из *Polarsteps* в разрабатываемое приложение перенесена идея представлять поездку как линию на карте с точками-событиями, а не как абстрактный список. Финансового контура и совместной работы у *Polarsteps* нет, обмен данными между устройствами идёт только через облако.

Готовые шаблоны маршрутов: Visit a City

Visit a City [25] предлагает каталог готовых маршрутов вида «1 день в Париже», «3 дня в Токио», который пользователь может скопировать к себе и слегка отредактировать. Главное достоинство — низкий порог входа: пользователь получает работоспособный план без самостоятельного планирования. Приложение оптимизировано под потребление готового, а не под создание собственного контента. Групповой работы и финансового модуля нет, в офлайн доступны только заранее скачанные шаблоны. Класс полезен как ориентир для будущей функции импорта типовых шаблонов поездок в Triloo.

Агрегаторы бронирований: TripIt

Подход с обратной стороны: собрать «паспорт поездки» автоматически из подтверждений бронирований на почте — перелёты, отели, аренда автомобиля, билеты. *TripIt* [21] складывает эти документы в один таймлайн, отслеживает изменения рейсов и присылает уведомления. От пользователя нужен минимум усилий, и для бизнес-путешественника это удобно. Зато детального планирования по дням здесь нет, с ROI-контентом и расходами группы такой «паспорт» не связан. Для разрабатываемого приложения класс полезен только как ориентир для будущего импорта подтверждений бронирований из почты.

B2B-инструменты для турагентов: Travify

Отдельный класс — приложения, рассчитанные не на самого путешественника, а на турагента, готовящего маршрут для клиента. *Travify* [26] позволяет агенту собрать многодневный план с фотографиями, описаниями и ROI, добавить

туда условия бронирования и сгенерировать PDF-документ для клиента. Качество выходного документа в этом классе высокое. Для частного пользователя класс закрыт сразу по нескольким причинам: подписочная B2B-модель оплаты, сложный интерфейс, отсутствие сценариев потребления плана самим путешественником в реальном времени поездки. Упомянут для полноты картины, как непосредственный конкурент к Triloo не относится.

Картографические приложения: Yandex Maps, Google Maps, 2GIS

По части поиска мест и навигации карты до сих пор остаются лучшими инструментами. POI-база с категориями, отзывами, фотографиями и часами работы; маршрутизация пешком, на машине и на общественном транспорте с учётом пробок; сценарий «что рядом со мной сейчас». *Yandex Maps* интегрирован с такси и общественным транспортом в России и поэтому естественный выбор для приложения, ориентированного на русскоязычного пользователя, особенно с учётом доступности SDK Yandex MapKit для нативной интеграции в Android-приложение [9]. *Google Maps* лидирует за пределами России и СНГ. *2GIS* силён в офлайн-режиме по российским городам.

Главный минус карт применительно к нашему сценарию — слабая структура плана. Закладки и пользовательские списки не равны расписанию по дням, совместное редактирование плана между несколькими участниками не поддерживается, финансового контура нет вовсе. Карта остаётся обязательным компонентом интегрированного помощника, но не заменяет планировщик. В Triloo карта используется как визуальная проекция плана поверх структурированных данных поездки, а не как основное хранилище.

Финансовые приложения: Splitwise, Tricount, Settle Up

Узкоспециализированный класс, отвечающий за расходы и долги в группе. *Splitwise* [27] позволяет ввести расход (платательщик, сумма, валюта, категория), разбить его на доли разными способами (поровну, точными долями, процентами, «только на этих»), посчитать балансы и предложить минимальный набор переводов для закрытия долгов. *Tricount* [28] и *Settle Up* [29] близки по логике, с акцентом на простой ввод и работу без регистрации. Модель расходов и расчёт «кто кому сколько» у *Splitwise* проработаны хорошо и взяты как эталон при проектировании финансового модуля Triloo.

Ограничения класса прямые: расходы изолированы от плана и POI, поэтому контекст «на что именно потрачено» теряется. Маршрут и логистика остаются вне системы. Офлайн-режим у всех трёх продуктов — это локальный ввод с последующей синхронизацией через сервер; обмена структурированными данными между устройствами напрямую нет.

AI-планировщики поездок: TripFlow и FlowTrip

Заметный сегмент последних двух-трёх лет — приложения, в которых план поездки автоматически собирается на основе входных параметров (направление, даты, интересы) и затем редактируется пользователем. *TripFlow* (iOS) — характерный представитель: по нескольким полям сервис формирует черновик маршрута, на главном экране сделан акцент на ближайшую поездку, на таймлайне дня между активностями отмечены блоки свободного времени, на карте маркеры мест дополнены нижним листом со списком активностей; отдельно есть AI-чат с квотой запросов и экран профиля для персонализации. *FlowTrip* (Android) ориентирован на групповой сценарий: на онбординге пользователь выбирает тип поездки (city trip, beach trip, festival и т. п.), затем AI собирает план с визуализацией процесса. В отдельном разделе уведомлений есть вкладки с групповыми голосованиями и приглашениями.

Из *TripFlow* в разрабатываемое приложение перенесены проработанный таймлайн с блоками свободного времени, связка «таймлайн, карта, список» в одном контуре и явное место AI в навигации по сценарию. У *FlowTrip* интересен акцент на групповой сценарий с голосованиями и приглашениями в отдельном разделе. Общие ограничения у обоих похожи на проблемы класса в целом: финансы не выделены отдельно и не связаны с планом, AI работает только онлайн и упирается в квоты стороннего сервиса, обмен данными между участниками без интернета не предусмотрен. Оба продукта заточены под онлайн-сценарий и не закрывают офлайн-обмен в группе.

Сводное сравнение и место Triloo

Сам обзор уже показал главное: каждый из разобранных классов решений закрывает только часть целевого сценария. Itinerary-планировщики (*Wanderlog*, *Roadtrippers*, *Sygyt Travel*) сильны в плане по дням и работе с POI, но не закрывают финансы и группу. Картографические приложения (*Yandex Maps*, *Google Maps*, *2GIS*) лидируют в навигации, но не дают структурированного расписания и совместной работы. Финансовые приложения (*Splitwise*, *Tricount*, *Settle Up*) закрывают расходы и мультивалютные сплиты, но изолированы от плана и маршрута. AI-планировщики (*TripFlow*, *FlowTrip*) собирают черновик плана и работают с группой, но не имеют финансового контура и не предусматривают обмена данными без сети. Из-за этого в типичной поездке пользователь вынужден держать одновременно четыре–шесть сервисов из разных классов и вручную сводить данные между ними.

Таблица 1 фиксирует наличие основных функциональных контуров целевого сценария у разобранных классов. Знак «+» означает, что соответствующий контур реализован как ключевая функция, знак «±» — частичная поддержка, знак «−» — контур отсутствует или находится вне фокуса продукта. Сразу видно, что ни одна из строк, кроме строки Triloo, не содержит плюсов во всех столб-

цах одновременно. Показательны два столбца: «офлайн-обмен» и «расходы со сплитами и мультивалютностью». На пересечении именно этих двух колонок ни одно из массовых решений не выходит в плюс. Это и есть отличительная часть постановки задачи разрабатываемого приложения.

Таблица 1 — Сравнение классов существующих решений по ключевым контурам сценария «поездка как проект»

Решение / класс	План по дням	Карта / POI	Маршрут / ETA	Группа, роли	Расходы, сплиты	Мульти-валютность	Офлайн-обмен
<i>Notion / Sheets</i> / мессенджеры	±	—	—	±	—	—	—
<i>Wanderlog</i>	+	+	±	±	—	—	—
<i>Roadtrippers</i>	+	+	+	±	—	—	—
<i>Sygie Travel</i>	+	+	±	—	—	—	±
<i>Polarsteps</i> (тревелог)	—	+	—	±	—	—	—
<i>Visit a City</i> (шаблоны)	±	+	—	—	—	—	±
<i>TripIt</i> (бронирования)	±	—	—	±	—	—	—
<i>Travefy</i> (B2B)	+	±	—	—	—	—	—
<i>Yandex Maps / Google Maps / 2GIS</i>	—	+	+	—	—	—	±
<i>Splitwise / Tricount / Settle Up</i>	—	—	—	±	+	+	—
<i>TripFlow</i> (AI itinerary, iOS)	+	+	±	±	—	—	—
<i>FlowTrip</i> (AI group planner, Android)	+	±	±	+	±	—	—
Triloo (цель работы)	+	+	+	+	+	+	+

Подходы из смежных областей

Помимо самих приложений, для постановки задачи полезно собрать подходы, накопленные в смежных областях. Они не специфичны для тревел-продуктов, но определяют технические решения, на которые опирается разрабатываемое приложение.

Принцип offline-first в мобильной разработке состоит в том, что локальное хранилище становится единственным источником истины для UI, а сеть рассматривается как опциональный канал получения обновлений и отправки изменений на сервер [1, 3]. Состояние UI всегда формируется из локальной базы, и интерфейс остаётся работоспособным при отсутствии связи. Подход развит в литературе по PWA и в современных мобильных фреймворках. В Android-разработке он естественно сочетается с Room в роли локального хранилища и Kotlin Flow в роли реактивного канала между источником данных и UI [10, 11, 12]. Разрабатываемое приложение реализует этот принцип целиком, без компро-

миссных «полу-онлайн» режимов.

Задача упорядочения точек посещения с учётом реальных расстояний близка к задаче коммивояжёра с временными окнами. Точное решение этой задачи NP-трудно [13], поэтому в массовых системах используются полиномиальные эвристики. Простейшая и наиболее известная из них — алгоритм ближайшего соседа: на каждом шаге к текущей точке добавляется ещё не посещённая точка, ближайшая по выбранной метрике [4]. На большинстве реальных входов алгоритм даёт решение в пределах 25–40% от оптимума [4, 5]. Для туристического сценария точности ближайшего соседа достаточно, особенно если в качестве дистанции брать не воздушное расстояние, а реальное время в пути из маршрутного API. В разрабатываемом приложении применён именно этот подход поверх данных из *OpenRouteService* и *Yandex Transport* (см. §3.1).

Для офлайн-обмена данными между устройствами через Bluetooth требуется защищать содержимое пакета от пассивного прослушивания эфира в зоне действия. Стандартное решение — шифрование AES в режиме Galois/Counter Mode (AES-GCM), описанное в спецификации NIST SP 800-38D [6]. GCM обеспечивает одновременно конфиденциальность (AES-256) и аутентификацию сообщения (тег длиной 128 бит): модификация любого байта пакета в эфире приводит к отказу проверки тега на приёмной стороне. Альтернативные режимы (CBC, CTR без аутентификации) требуют отдельной HMAC-обвязки и менее удобны в реализации.

Bluetooth Classic с профилем RFCOMM — это потоковая абстракция поверх беспроводной L2CAP-сессии, удобная для передачи произвольных по размеру пакетов между двумя устройствами [7, 8]. Bluetooth Low Energy (BLE) и его GATT-характеристики оптимизированы под короткие сообщения (десятки–сотни байт), а пакет данных поездки в типовом сценарии весит сотни килобайт. По этой причине в разрабатываемом приложении выбран Bluetooth Classic RFCOMM, а BLE рассмотрен и отброшен. Альтернативные транспорты ближайшего радиуса — Wi-Fi Direct и mesh-сети поверх UWB — на текущем этапе не рассматриваются.

При обмене данными между устройствами без центрального арбитра неизбежно возникают конфликты редактирования одной и той же сущности на двух устройствах. В литературе выделяются два основных подхода. Last-Write-Wins (LWW) разрешает конфликт по временной метке последнего обновления и реализуется на уровне отдельной сущности или поля. CRDT-репликация (Conflict-free Replicated Data Types) разрешает конфликты структурно, исходя из коммутативной алгебры операций над данными [14]. CRDT даёт более естественное поведение при одновременном редактировании, но требует существенно более сложной реализации и пересмотра доменной модели. В приложении Triloo выбран LWW на уровне сущности по полю `updatedAt`; переход к CRDT отнесён к перспективам развития.

Для сценария быстрого ввода расходов «сфотографировал — добавил» полезно on-device OCR изображений чеков. Современные Android-устройства

поддерживают это через Google ML Kit Text Recognition [15] — библиотеку, выполняющую распознавание текста полностью на устройстве, без отправки изображений на сервер. Из полученного текста типовой чек поддаётся достаточно надёжному разбору регулярными выражениями: итоговая сумма, валюта (детекция символов □, €, \$ и трёхбуквенных кодов ISO 4217), дата и время покупки. Подход с устранением сторонних сервисов из обработки чеков согласуется с требованиями офлайн-сценария и приватности.

Готового решения, в котором собрана связка «план + карта + расходы + группа» и офлайн-обмен структурированными данными между участниками, в массовых тревел-продуктах нет. Это и обосновывает разработку собственного приложения. Уточнённая постановка задачи формулируется в §1.4 после определения объекта и методов разработки в §1.3.

1.3. Объект и методы разработки

Объектом разработки выступает процесс подготовки и сопровождения групповых путешествий с использованием мобильных программных средств. Объект охватывает действия пользователя на всех стадиях поездки: от первоначального планирования по дням и поиска мест, через ведение бюджета и согласование решений в группе, до получения и применения данных поездки в условиях нестабильной или отсутствующей сети.

Предмет разработки — программные средства и методы интегрированного мобильного помощника путешественника на платформе Android, обеспечивающие единую среду планирования по дням, поиска мест и навигации, мультивалютного учёта расходов с расчётом долгов, групповой координации, а также офлайн-обмен структурированными данными поездки между устройствами по каналу Bluetooth с симметричным шифрованием. Сужение от объекта к предмету выражается формулой «исследуем процесс подготовки путешествия на предмет того, как собрать его в одно мобильное приложение и снять зависимость от сети при обмене данными в группе».

Реальный объект разработки — мобильное приложение Triloo для платформы Android. Приложение написано на Kotlin 2.0.21, использует Jetpack Compose 2024.09 в качестве декларативного UI-фреймворка, Room 2.7.1 в качестве локального хранилища, Hilt 2.56 в качестве контейнера зависимостей, Kotlin Coroutines 1.9 и Kotlin Flow в качестве модели асинхронности и реактивных потоков [10, 11, 12]. Минимальная поддерживаемая версия Android — 8.0 (API 26), целевая — Android 15 (API 36). Картографическая часть построена на Yandex MapKit 4.33 [9], автодополнение мест — на двух клиентах (Yandex Geosuggest и Geoapify Places [16]), маршрутизация — на OpenRouteService [17] и Yandex Transport, OCR чеков — на Google ML Kit Text Recognition 16.0 [15]. Подсистема офлайн-обмена использует Bluetooth Classic с профилем RFCOMM [8] и шифрование AES-256 в режиме GCM [6]. Онлайн-синхронизация работает с собственным backend на Node.js

(Express + better-sqlite3). Структурно приложение разделено на четыре слоя: UI, ViewModel, репозитории, источники данных. Разделение соответствует шаблону MVVM и принципам Clean Architecture (см. главу 2).

Информационная база разработки включает официальную документацию платформы Android и набора Jetpack [10, 11, 12], спецификацию языка Kotlin, спецификацию Bluetooth Core 5.4 [7] и описание профиля RFCOMM в спецификации Bluetooth SIG, стандарт NIST SP 800-38D «Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC» [6], работы по эвристикам решения задачи коммивояжера [4, 5, 13], материалы по offline-first-архитектурам и репликации данных в мобильных приложениях [1, 2, 3, 14], ГОСТ 7.32-2017 [30] и ГОСТ Р 7.0.99-2018 [31] в части оформления настоящей работы, Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» в части обращения с персональными данными участников поездки [32], а также открытую документацию используемых сторонних API (*Yandex MapKit*, *Yandex Geosuggest*, *Geoapify Places*, *OpenRouteService*, *Open Exchange Rates*).

В работе использованы следующие методы.

1. Системный анализ предметной области с выделением слоёв сценария поездки (расписание, навигация, бюджет, группа, сеть) и пересечений между ними. Результаты закреплены в §1.1 и в обобщённой постановке задачи.
2. Сравнительный анализ существующих решений по фиксированному набору критериев (план по дням, карта и POI, маршрут и ETA, группа и роли, расходы и сплиты, мультивалютность, офлайн-обмен) с целью выявить незакрытую нишу. Результаты собраны в §1.2 и в табл. 1.
3. Объектно-ориентированное проектирование доменной модели поездки и её сущностей (Trip, TripDay, Place, Expense, Participant, DeletionLog) с явными ссылочными связями и временными метками для слияния.
4. Шаблон MVVM для построения клиентского слоя и принципы Clean Architecture для разделения ответственностей между UI, доменом и источниками данных.
5. Эвристика ближайшего соседа для оптимизации порядка обхода точек посещения поверх реальных дистанций, получаемых из маршрутного API [4, 5].
6. Симметричное шифрование AES-256 в режиме GCM для защиты содержимого пакета данных поездки, передаваемого через Bluetooth-канал [6].
7. Last-Write-Wins на уровне сущности по полю updatedAt как стратегия разрешения конфликтов при слиянии пакетов офлайн-обмена и при онлайн-синхронизации с backend.

8. Модульное тестирование на JVM (JUnit 4, Robolectric) для верификации доменной логики (расчёт балансов, оптимизация маршрута, парсер чеков, слияние пакетов Triloo Relay) без эмулятора.
9. Эксперимент с приложением: наблюдение за поведением приложения в типовых сценариях, замер поведения в офлайн-режиме, проверка идемпотентности повторного слияния пакетов, сравнение длины оптимизированного маршрута с тривиальным порядком (см. главу 4).

Сами по себе перечисленные методы на оригинальность не претендуют. Вклад работы — в их согласованной интеграции в одном мобильном приложении с офлайн-обменом данными между устройствами.

1.4. Уточнённые требования к разрабатываемому приложению Triloo

С учётом характеристики задачи (§1.1), обзора существующих решений (§1.2) и определения объекта и методов разработки (§1.3) окончательная постановка формулируется ниже. Из неё явно исключены сценарии, отнесённые к дальнейшему развитию: продвинутая AI-генерация плана, mesh-сети и многошаговый офлайн-обмен через цепочку устройств, поле-к-полю CRDT-репликация, полностью клиент-серверный сценарий без локального хранилища.

Цель работы

Цель работы — разработать мобильное приложение Triloo для платформы Android, дающий пользователю единую среду подготовки и сопровождения поездки и позволяющий обмениваться структурированными данными поездки между устройствами без интернета.

Задачи работы

Для достижения цели в работе решаются следующие задачи.

1. Спроектировать многоуровневую offline-first-архитектуру клиента с явными уровнями ответственности (UI, ViewModel, репозитории, источники данных), моделью предметной области и принципами работы с внешними сервисами так, чтобы ключевые сценарии работали без сети.
2. Спроектировать локальное хранилище и доменную модель поездки: схему сущностей «поездка, день, место, расход, участник» с журналом удалений и поддержкой миграций; реализовать её на Room.
3. Реализовать планирование по дням и работу с местами: автоматическое создание дней по диапазону дат поездки, добавление мест с автодополнением и геокодингом, хранение расширенных атрибутов места (категория,

координаты, рейтинг, часы работы), отметку «посещено», перестановку точек в дне.

4. Подключить картографию и маршрутизацию: маркеры мест и участников на карте, построение полилинии маршрута через внешние маршрутные сервисы, расчёт расстояния и оценочного времени в пути в разных режимах перемещения.
5. Реализовать эвристическую оптимизацию порядка посещения точек по алгоритму ближайшего соседа по геодезическим расстояниям (формула гаверсинусов); реальные дорожные дистанции из маршрутных API использовать для построения и оценки итогового маршрута.
6. Реализовать финансовый модуль: ввод расходов с категориями, плательщиками, сплитами и валютой, мультивалютную конвертацию по курсам с локальным кешем, расчёт балансов участников и минимального набора переводов для закрытия долгов.
7. Реализовать групповые сценарии: приглашение в поездку по 6-символьному коду и QR-коду, роли участников (OWNER, ADMIN, MEMBER), геошаринг через Foreground Service.
8. Разработать и реализовать подсистему офлайн-обмена данными Triloo Relay: сбор переносимого пакета поездки, обмен по Bluetooth Classic с симметричным шифрованием AES-256-GCM, слияние данных на принимающей стороне с учётом журнала удалений.
9. Реализовать дополнительные прикладные сценарии: распознавание чеков (on-device OCR через Google ML Kit), рекомендации жилья по геоданным, экспорт локальных данных, уведомления-напоминания о ближайших активностях поездки.
10. Подготовить почву для облачной синхронизации: сетевой клиент к собственному backend по HTTPS с ролевым контролем доступа.
11. Провести экспериментальную проверку приложения: корректность слияния данных при офлайн-синхронизации, качество маршрута на выходе оптимизатора, поведение приложения в типовых офлайн-сценариях (глава 4).

Роли пользователей и ключевые варианты использования приложения сведены на диаграмме вариантов использования (рис. 2); ниже они детализированы функциональными требованиями.

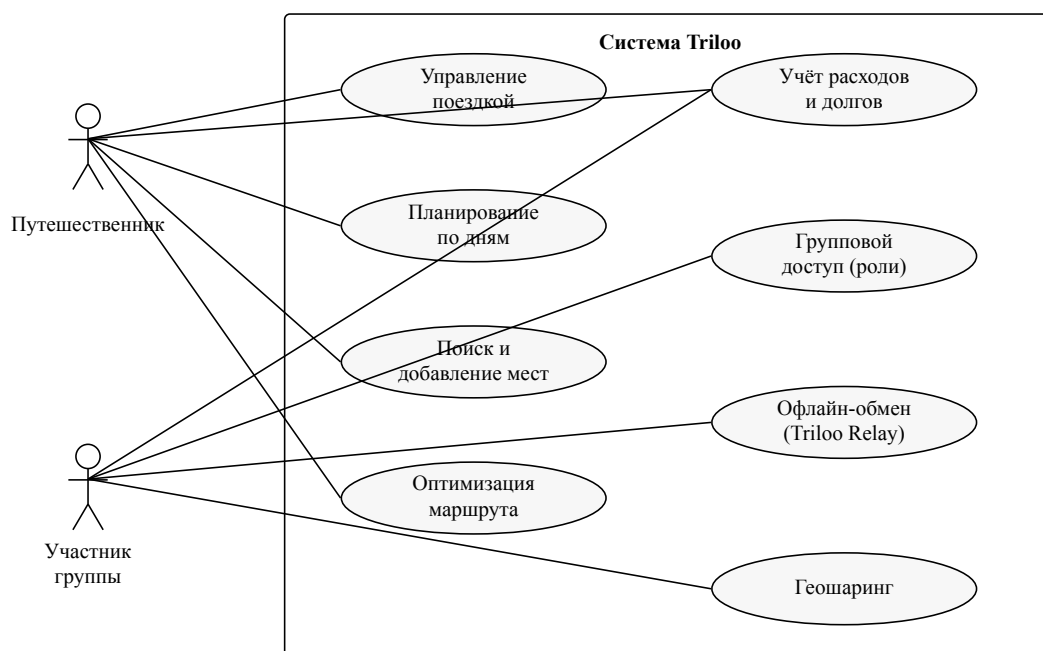


Рис. 2. UML-диаграмма вариантов использования Triloo

Функциональные требования

Функциональные требования к приложению сведены в табл. 2.

Таблица 2 — Функциональные требования к приложению Triloo

Код	Требование
F1	Управление поездками: создание, просмотр, редактирование и удаление поездки с обязательными полями (название, направление, даты, базовая валюта) и опциональными (бюджет, место проживания, обложка).
F2	Планирование по дням: автоматическое разбиение поездки на дни по диапазону дат, добавление мест в день, перестановка точек в пределах дня, отметка «посещено».
F3	Поиск и добавление мест через строку с автодополнением (Geoapify Places, Yandex Geosuggest); извлечение координат и атрибутов (название, адрес, категория, рейтинг, часы работы, фото); ручной ввод заметок, времени и длительности.
F4	Карта и маршруты: маркеры мест и участников на Yandex MapKit, полилиния маршрута через OpenRouteService и Yandex Transport, расстояние и оценочное время в пути в разных режимах перемещения.
F5	Оптимизация порядка посещения: эвристическое улучшение порядка точек алгоритмом ближайшего соседа по геодезическим расстояниям (формула гаверсинусов), с фиксацией стартовой точки; реальные дорожные дистанции из маршрутных API используются для построения и оценки итогового маршрута.
F6	Учёт расходов: добавление, редактирование и удаление расходов с полями описания, суммы, валюты, категории, плательщика, даты и заметок; сводные показатели по поездке и категориям.
F7	Мультивалютность: перевод расходов в базовую валюту по курсам из внешнего API с локальным кешем; предупреждение пользователя, если кеш курсов устарел.

Продолжение таблицы 2

Код	Требование
F8	Сплиты и расчёт долгов: разбиение расхода в режимах PAYER_ONLY, EQUAL и EXACT (режимы PERCENTAGE и SHARES предусмотрены моделью данных и отнесены к дальнейшему развитию); расчёт парных балансов и упрощённый набор переводов для закрытия всех долгов.
F9	Групповые поездки: приглашение по 6-символьному коду и QR-коду, роли OWNER / ADMIN / MEMBER, разграничение прав на изменение данных поездки.
F10	Геошаринг: отображение позиций участников на карте по явному согласию, обновление координат через Foreground Service с ограничениями по частоте и точности.
F11	Офлайн-обмен данными (Triloo Relay): обмен пакетом поездки по Bluetooth Classic RFCOMM, шифрование AES-256-GCM, слияние входящего пакета по правилу Last-Write-Wins с учётом журнала удалений.
F12	Онлайн-синхронизация: отправка снапшота поездки на собственный backend по HTTPS с ролевым контролем доступа; получение и слияние изменений с сервера.
F13	Распознавание чеков: on-device OCR изображения чека через Google ML Kit, извлечение суммы, валюты, даты и времени покупки с предзаполнением формы расхода.
F14	Рекомендации жилья: ранжирование объектов размещения по эвристикам (расстояние от ключевой точки, попадание в бюджет, рейтинг, звёздность) поверх Geoapify Places.
F15	Настройки и экспорт: тема оформления (светлая, тёмная, системная), локализация (русский и английский), управление уведомлениями, экспорт локальных данных поездки.

Нефункциональные требования

N1. Совместимость: поддержка Android 8.0 и выше (minSdk 26), целевая платформа — Android 15 (API 36), поддержка различных размеров экранов мобильных устройств.

N2. Offline-first: ключевые сценарии (просмотр и редактирование поездки, добавление мест и расходов, расчёт балансов, обмен данными по Bluetooth) работают без интернета. Сеть требуется только тому, что объективно от неё зависит: картографические тайлы, автодополнение мест, маршрутные API, курсы валют, онлайн-синхронизация.

N3. Производительность: плавная прокрутка списков на устройствах среднего класса; работа с базой данных и сетью не блокирует UI-поток. У геошаринга разумные ограничения по частоте обновлений (одно обновление в 30–60 секунд) и пороговое смещение позиции для записи.

N4. Надёжность: корректная работа без сети и при нестабильном Bluetooth; сохранность локальных данных при перезапуске приложения и обновлении схемы базы; идемпотентность повторного применения пакета Triloo Relay (повторный приём того же пакета не дублирует записи).

N5. Безопасность и приватность: доступ к геолокации и Bluetooth только

по явному разрешению пользователя; шифрование данных в Triloo Relay (AES-256-GCM); связь с backend только по HTTPS; ключи сторонних сервисов хранятся через Gradle Secrets и попадают в BuildConfig на этапе сборки, в репозитории остаются только плейсхолдеры. Обработка персональных данных участников поездки (имя, фотография профиля, координаты при геошаринге) согласуется с требованиями Федерального закона № 152-ФЗ «О персональных данных» [32]: данные не передаются третьим лицам, геошаринг включается явным согласием каждого участника, локальное хранилище доступно только владельцу устройства.

№6. Поддерживаемость: разделение по слоям (UI, ViewModel, репозитории, источники данных); внедрение зависимостей через Hilt; реактивные потоки на Kotlin Coroutines и Flow; модульные тесты на JVM для ключевой логики (расчёт балансов, оптимизация маршрута, слияние пакетов Triloo Relay, парсер чеков).

Ограничения и допущения

Для работы карт и поиска мест нужны ключи внешних API (Yandex MapKit, Geoapify, OpenRouteService). Их получение относится к развёртыванию приложения и не входит в состав работы; в исходном коде хранятся только плейсхолдеры в файле `local.defaults.properties`.

Для Triloo Relay нужны включённый Bluetooth у обоих устройств и их физическая близость в радиусе действия Bluetooth Classic (типично 10–30 метров). Wi-Fi Direct, mesh-сети и многошаговая ретрансляция через цепочку устройств на текущем этапе не рассматриваются.

AI-функциональность (автоматическая генерация плана, AI-рекомендации) обозначена как направление развития. В текущей версии используется только эвристическая логика и обращение к внешним сервисам через документированные API.

Конфликты при онлайн-синхронизации сейчас разрешаются по правилу Last-Write-Wins на уровне сущности. Поле-к-полю слияние и CRDT-репликация отнесены к дальнейшему развитию.

На текущем этапе допускается использование мок-данных для ускорения проверки сценариев с обязательной заменой на реальные интеграции до защиты.

Критерии качества решения

Качество решения оценивается по следующим критериям, проверяемым в главе 4.

Покрытие функциональных требований. Все требования F1–F15 имеют рабочую реализацию в приложении и могут быть продемонстрированы на типовом сценарии «подготовка трёхдневной поездки в Сеул для двух участников с расходами в нескольких валютах».

Корректность слияния пакетов офлайн-обмена. Повторное применение одного и того же пакета на стороне приёмника не приводит к дублированию записей; ранее удалённые сущности не возвращаются после следующей синхронизации; при одновременном изменении одной и той же сущности на двух устройствах принимается запись с более поздним временем `updatedAt`.

Качество маршрута на выходе оптимизатора. Длина маршрута, построенного эвристикой ближайшего соседа по геодезическим расстояниям (формула гаверсинусов), на типовых наборах входов (5–15 точек) ниже длины маршрута в исходном порядке добавления точек пользователем; отклонение от оптимума на сравниваемых наборах укладывается в типичные для алгоритма пределы [4, 5].

Поведение в офлайн-режиме. Ключевые сценарии (просмотр поездки, добавление расхода, расчёт балансов, обмен пакетом по Bluetooth) выполнимы без интернета и не приводят к падению приложения; нестабильное Bluetooth-соединение восстанавливается автоматически в пределах разумных таймаутов.

Безопасность канала Triloo Relay. Пакет данных поездки, перехваченный в Bluetooth-эфире, не поддаётся прочтению без знания общего секрета сессии; модификация хотя бы одного байта пакета приводит к отказу проверки тега AES-GCM на приёмнике.

Сформулированные таким образом цель, задачи, требования и критерии качества определяют то, как устроена архитектура приложения Triloo, описываемая в главе 2.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ TRILOO

Цель, задачи и требования из §1.4 определяют, что приложение должно делать; настоящая глава описывает, как оно для этого устроено. Архитектура Triloo подчинена двум сквозным требованиям: ключевые сценарии должны работать без сети (N2), а подсистему обмена данными нужно строить так, чтобы один и тот же доменный слой обслуживал и локальную работу, и офлайн-обмен, и онлайн-синхронизацию (N6, задачи 1 и 8). Из этих двух требований вытекают остальные проектные решения, и глава излагает их от общей схемы к частностям: многоуровневая offline-first-архитектура клиента (§2.1), доменная модель поездки и локальное хранилище (§2.2), интеграционный слой со сторонними сервисами (§2.3) и подсистема офлайн-обмена Triloo Relay (§2.4). Алгоритмы, реализующие отдельные подсистемы, — оптимизация маршрута, расчёт долгов, протокол обмена по шагам, онлайн-синхронизация — вынесены в главу 3, экспериментальная проверка — в главу 4.

2.1. Многоуровневая offline-first-архитектура клиента

В основу клиентской части положен шаблон MVVM (Model–View–ViewModel) в сочетании с принципами чистой архитектуры (Clean Architecture) [33, 34]. Выбор продиктован двумя соображениями. Во-первых, требование offline-first (N2) проще всего выполнить, когда состояние интерфейса формируется не из ответов сети, а из локального хранилища; для этого нужен слой, прозрачно для UI подменяющий сетевой канал локальной базой. Во-вторых, требование поддерживаемости и тестируемости (N6) предполагает, что доменная логика — расчёт балансов, оптимизация маршрута, слияние пакетов — не зависит от Android-фреймворка и проверяется юнит-тестами на JVM без эмулятора. Оба соображения приводят к разделению на слои с однонаправленными зависимостями: вышележащий слой знает о нижележащем, но не наоборот.

Клиент разделён на четыре слоя (рис. 3). Слой UI образован экранами на Jetpack Compose [11], сгруппированными по функциональным пакетам (ui/trips, ui/tripdetails, ui/budget, ui/grouptrips, ui/relay, ui/settings и др.). Экран — это набор @Composable-функций, которые не хранят состояния, а лишь отображают пришедшее от ViewModel и сообщают о действиях пользователя обратными вызовами. Слой представления образуют ViewModel'и, помеченные @HiltViewModel и получающие зависимости через конструктор; каждый из них собирает состояние экрана в один неизменяемый объект и публикует его как StateFlow (например, TripListViewModel, TripDetailsViewModel, BudgetRouterViewModel). Доменный слой — репозитории (TripRepository, ExpenseRepository, RelayRepository, OnlineSyncRepository и др.), которые инкапсулируют

операции над данными и не знают ни о Compose, ни о деталях транспортов. Нижний слой — источники данных: локальная база Room с объектами доступа (DAO), сетевые клиенты Retrofit/OkHttp и хранилище настроек DataStore.

Связь между слоями построена на реактивных потоках Kotlin Flow [36]. DAO Room возвращают данные не разовым запросом, а как Flow, повторно эмитящий значение при каждом изменении таблицы. Репозиторий комбинирует такие потоки и отдаёт их ViewModel, ViewModel преобразует их в StateFlow состояния экрана, а Compose подписывается на него через `collectAsStateWithLifecycle()` — сбор автоматически приостанавливается, когда экран уходит с переднего плана. Так, состояние экрана деталей поездки в `TripDetailsViewModel` собирается оператором `combine` из потоков `observeTripById`, `observeTripDays`, `observePlacesByTrip`, `observeParticipants` и `observeExpensesByTrip`; добавление места или расхода меняет соответствующую таблицу Room, и обновлённое состояние доходит до экрана без явного запроса на пересчитывание. Локальная база становится единственным источником истины для интерфейса.

Этот же принцип определяет порядок записи. Пользовательское действие сначала фиксируется в Room и лишь затем, отдельной фоновой задачей, отправляется во внешние каналы. Так, `TripRepository.createTrip` вставляет поездку и её дни в локальную базу и только после этого вызывает `onlineSyncRepository.syncTripAsync`, выполняемый в области с `SupervisorJob` на `Dispatchers.IO` и не блокирующий ни UI, ни сам факт создания поездки. Если сети нет, локальная запись уже состоялась, а синхронизация произойдёт при следующей возможности. Все обращения к базе и сети вынесены из главного потока на `Dispatchers.IO`; на главном потоке остаётся только то, что обязано там быть, например вызовы `Yandex MapKit`, требующие UI-потока.

Сборка графа зависимостей возложена на Hilt [35]. Модуль `DatabaseModule` предоставляет экземпляр базы и объекты DAO, `NetworkModule` — сетевые клиенты, `AuthModule` — реализацию авторизации, `RouteModule` связывает интерфейсы маршрутизации и поиска мест с их реализациями. Репозитории и ViewModel получают зависимости через конструктор, и это делает доменную логику тестируемой: в тесте вместо реального репозитория или сетевого клиента подставляется заглушка. Версии ключевых библиотек, на которых построен клиент, приведены в табл. 3.

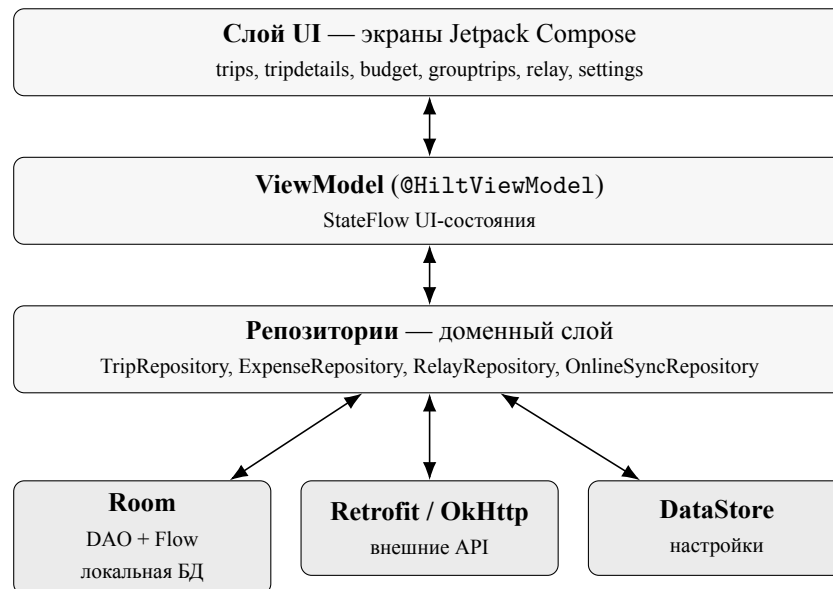


Рис. 3. Многоуровневая offline-first-архитектура клиента Triloo

Состав компонентов клиента и зависимости между ними показаны на рис. 4. Компоненты сгруппированы по слоям, а зависимости направлены сверху вниз: интерфейс зависит от ViewModel, тот — от репозитория, репозитории — от источников данных (Room, сеть, DataStore, подсистема Relay); обращение к собственному backend изолировано в сетевом компоненте.

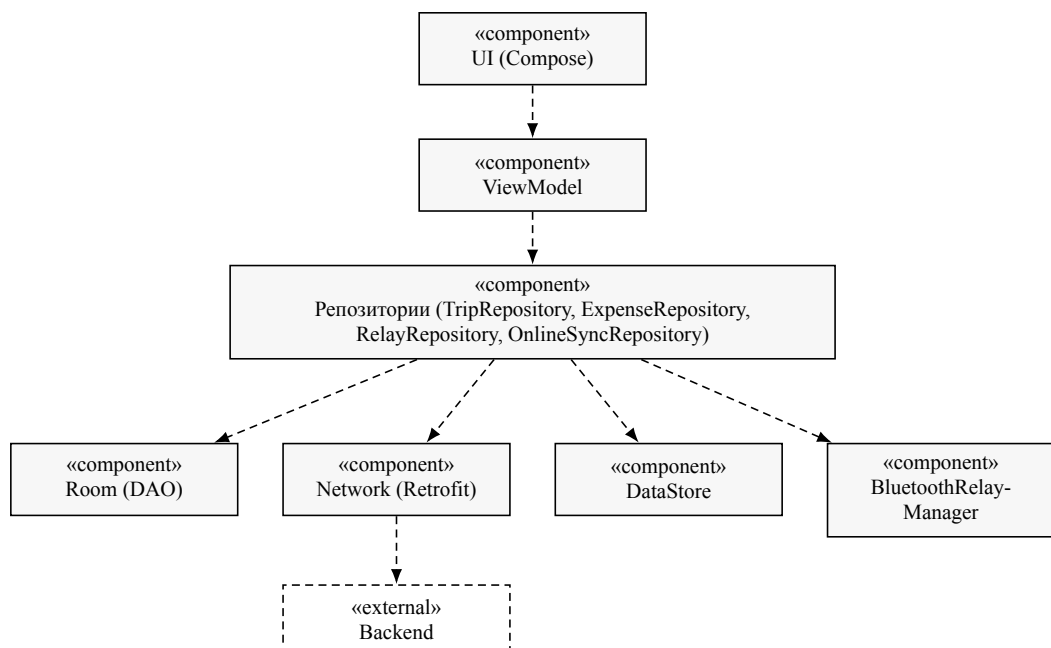


Рис. 4. UML-диаграмма компонентов клиента Triloo (зависимости показаны пунктиром)

Таблица 3 — Основные технологии и библиотеки клиентской части Triloo

Компонент	Версия	Назначение
Kotlin	2.0.21	Язык реализации
Jetpack Compose (BOM)	2024.09	Декларативный UI
Room	2.7.1	Локальная база данных (источник истины)
Hilt	2.56	Внедрение зависимостей
Kotlin Coroutines / Flow	1.9	Асинхронность и реактивные потоки
Retrofit / OkHttp	2.11 / 4.12	HTTP-клиенты к внешним API
Gson	2.11	Сериализация JSON, в т.ч. пакета Relay
DataStore	1.1.6	Хранилище пользовательских настроек
Yandex MapKit	4.33	Карта, поиск мест, маршруты
ML Kit Text Recognition	16.0	Распознавание чеков на устройстве
Android SDK	minSdk 26 / target 36	Целевая платформа (Android 8.0–15)

2.2. Доменная модель поездки и локальное хранилище

Доменная модель отражает структуру предметной области, зафиксированную в §1.1: поездка состоит из дней, дни наполнены местами, к поездке привязаны расходы и участники. Модель образована восемью сущностями (рис. 5): Trip (поездка), TripDay (день), Place (место), Expense (расход) и вспомогательная ExpenseSplit (доля участника в расходе), Participant (участник), CurrencyRate (кэш валютного курса) и DeletionLog (журнал удалений). Сущности хранятся в локальной базе Room [12]; каждая описана как @Entity, связи между ними заданы внешними ключами.

Первичные ключи поездки, дня, места и расхода — строковые идентификаторы UUID, генерируемые на устройстве в момент создания записи, а не автоинкрементные счётчики базы. Это решение продиктовано офлайн-обменом: две поездки или два расхода могут быть созданы независимо на разных устройствах, не имеющих общего источника нумерации, и при последующем слиянии их идентификаторы не должны сталкиваться. UUID снимает потребность в централизованном генераторе ключей и делает запись самодостаточной — её можно перенести на другое устройство и вставить без перенумерации ссылок.

Каждая доменная сущность несёт временные метки createdAt и updatedAt (в миллисекундах). Поле updatedAt — опора стратегии разрешения конфликтов Last-Write-Wins (§1.2, [14]): при слиянии двух версий одной записи принимается та, у которой метка позже. Метка обновляется при любом изменении сущности в репозитории, поэтому к моменту обмена данными у каждой записи есть актуальное время последней правки, по которому сравниваются конкурирующие версии.

Связи «поездка — день — место» и «поездка — расход» заданы внешними ключами с каскадным удалением (`onDelete = CASCADE`): удаление дня убирает его места, удаление поездки — её дни, места, расходы и доли. Для ускорения частого запроса «все места поездки» в сущность `Place` добавлено денормализованное поле `tripId` помимо ссылки на день `tripDayId`; это сознательное отступление от нормальной формы в пользу простоты и скорости выборки, типичное для встраиваемых баз на мобильных устройствах. Участник `Participant` и доля `ExpenseSplit` имеют составные первичные ключи (`tripId + userId` и `expenseId + userId` соответственно), отражающие их природу как связей «многие ко многим» с дополнительными атрибутами.

База `TrilooDatabase` находится в третьей версии схемы, схема экспортируется в каталог `app/schemas` для контроля совместимости. Типы, не отображаемые в `SQLite` напрямую, переводятся конвертерами: `LocalDate` сериализуется в строку формата ISO-8601, перечисления (`TripStatus`, `ParticipantRole`, `PlaceCategory`, `ExpenseCategory`, `SplitType`, `RelayEntityType`) — в имена констант, карта долей `splitAmounts` — в JSON. Эволюция схемы оформлена миграциями: `MIGRATION_1_2` добавила участникам метки времени и создала таблицу журнала удалений, `MIGRATION_2_3` — координаты центра направления поездки. Применение миграций, а не пересоздание базы, сохраняет локальные данные пользователя при обновлении приложения (N4).

Отдельного внимания заслуживает сущность `DeletionLog`. В обычной базе удаление записи не оставляет следа, но при офлайн-обмене именно отсутствие следа порождает ошибку: если одно устройство удалило расход, а второе об этом ещё не знает, то при следующем слиянии второе устройство «вернёт» удалённую запись как новую. Чтобы этого не происходило, каждое удаление доменной сущности фиксируется записью в `DeletionLog` с указанием типа сущности (`entityType`), её идентификатора (`entityId`), времени удаления (`deletedAt`) и устройства-источника (`deviceId`). Журнал удалений переносится вместе с данными поездки и при слиянии применяется как самостоятельный артефакт: удалённая на одном устройстве запись остаётся удалённой и на других. Вынесение журнала удалений в отдельную сущность доменной модели — один из элементов новизны работы (см. введение); как он используется при слиянии, описано в §2.4.

Пользовательские настройки приложения хранятся отдельно от доменных данных — в `DataStore` (`AppSettingsRepository`): тема оформления, базовая валюта по умолчанию, язык интерфейса, разрешение уведомлений и флаг синхронизации. Эти значения не участвуют в обмене между устройствами и не нуждаются в реляционной модели, поэтому для них выбрано хранилище «ключ — значение», а не таблица `Room`.

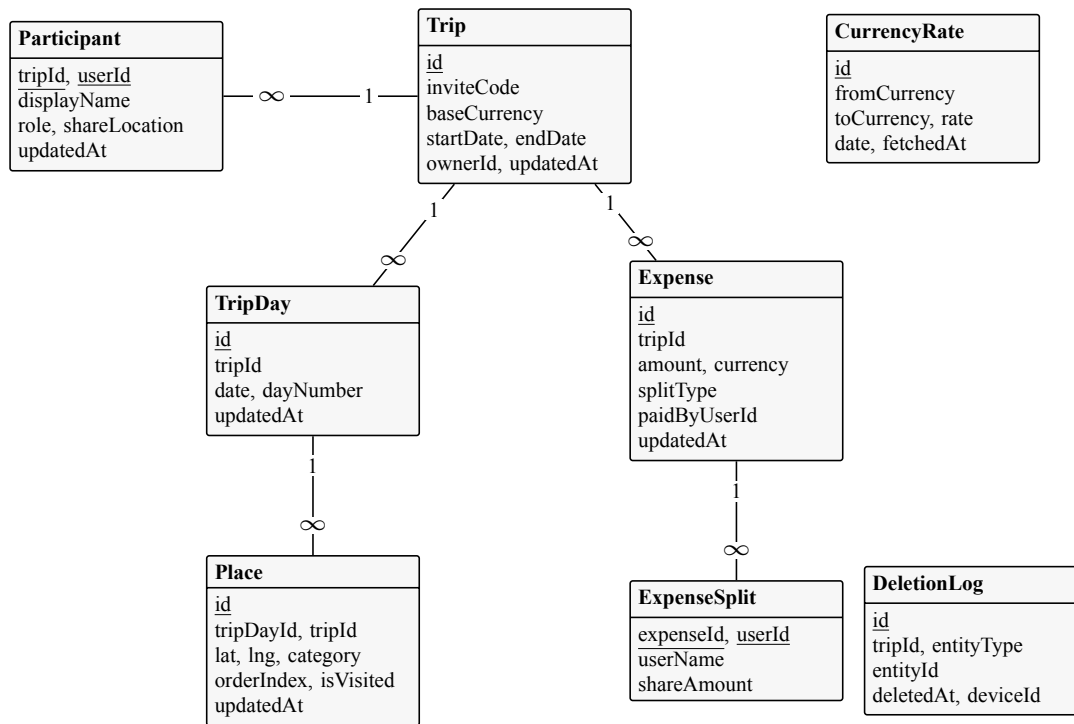


Рис. 5. ER-схема доменной модели и локального хранилища Triloo (подчёркнуты ключевые атрибуты)

2.3. Интеграционный слой со сторонними сервисами

Часть функций приложения объективно зависит от внешних сервисов: тайлы карты, автодополнение адресов, построение маршрута, курсы валют, онлайн-синхронизация. Чтобы эта зависимость не нарушала offline-first-принцип (N2), интеграционный слой построен по правилу: внешний сервис скрыт за репозиторием или сервисным классом, доменный слой и UI обращаются к нему через узкий интерфейс и не знают деталей сетевого протокола, а недоступность сервиса деградирует лишь ту функцию, которой он нужен, не затрагивая локальное ядро. Карту нельзя загрузить без сети — но просмотр плана, добавление расхода и расчёт долгов от карты не зависят и остаются работоспособными.

Сетевые клиенты создаёт модуль `NetworkModule`. Для каждого внешнего API заведён именованный экземпляр `Retrofit` [37] поверх общего `OkHttpClient` с таймаутами 20/45/20 секунд; всего таких экземпляров семь — по числу обслуживаемых сервисов (табл. 4). Помимо них приложение использует два нативных SDK, не работающих через `Retrofit`: `Yandex MapKit` (карта, поиск, маршруты) [9] и `Google ML Kit` (распознавание чеков на устройстве) [15]. Отдельный случай — клиент `Geoarify` [16]: для него заведён выделенный `OkHttpClient`, в котором отключено `gzip`-сжатие ответа и форсирован протокол `HTTP/1.1`, что обходит ошибку разрыва сжатого потока на промежуточном узле. Такие частные настройки изолированы в сетевом модуле и не протекают в доменный слой.

Там, где это уместно, ответы внешних сервисов кэшируются, что дополнительно снижает зависимость от сети. Полученные курсы валют сохраняются

в таблице `CurrencyRate` и переиспользуются при конвертации расходов; если кэш устарел, пользователь получает явное предупреждение (N7). Результаты поиска мест и ссылки на фотографии кэшируются в памяти сервиса `PlacesService`. Ключи доступа к внешним API не хранятся в репозитории: они подставляются на этапе сборки из локального файла свойств в поля `BuildConfig` через механизм `Gradle Secrets`, а в коде остаются только плейсхолдеры (N5, §1.4). Доступность картографического представления вынесена в сборочный флаг `APP_MARKIT_VIEW_ENABLED`.

Изоляция доведена до уровня интерфейсов там, где у функции может быть несколько реализаций. Модуль `RouteModule` связывает интерфейсы `RouteOptimizer` (оптимизация порядка точек), `MapRouteProvider` (построение маршрута) и `NearbyPlacesProvider` (поиск ближайших мест) с конкретными реализациями; доменный слой зависит от интерфейса, а не от провайдера, поэтому маршрутный сервис можно заменить, не трогая вызывающий код, и подменить заглушкой в тесте. Аналогично устроена авторизация: `AuthModule` выбирает реализацию `AuthRepository` по доступности — собственный backend, иначе `Firebase`, иначе локальная заглушка для разработки. Перечень сервисов, обслуживающих их модулей и поведения без сети сведён в табл. 4.

Таблица 4 — Интеграционный слой: внешние сервисы и поведение без сети

Сервис	Модуль-клиент	Тип	Назначение	Без сети
Yandex MapKit	YandexRouter, PlacesService	SDK	Карта, поиск, маршруты	недоступно
Yandex Geosuggest	PlacesService / GeosuggestApi	Retrofit	Автодополнение адресов	недоступно
Geoapify Places	PlacesService, Accommodation-Recommendation-Service	Retrofit	Поиск мест, рекомендации жилья	кэш в памяти
OpenRoute-Service	YandexRouter / OpenRouteService-Api	Retrofit	Маршруты (резерв)	недоступно
Open Exchange Rates	ExpenseRepository / CurrencyApi	Retrofit	Курсы валют	кэш в Room
Google ML Kit	ReceiptOcrService	SDK	OCR чеков (на устройстве)	доступно
Gemini	OpenAiService / OpenAiApi	Retrofit	AI-подсказки (развитие)	недоступно
Собственный backend	OnlineSync-Repository, BackendAuth-Repository	Retrofit	Авторизация, онлайн-синхронизация	недоступно

2.4. Подсистема офлайн-обмена Triloo Relay

Triloo Relay решает задачу, которую, как показал обзор §1.2, не закрывает ни один массовый тревел-продукт: передать структурированные данные поездки с одного устройства на другое без сети и корректно слить их с локальной базой принимающей стороны (F11, задача 8). Подсистема построена из пяти компонентов с разделёнными обязанностями: `BluetoothRelayManager` отвечает за транспорт — обнаружение устройств, установление соединения и обмен кадрами протокола; `RelayRepository` — за прикладную логику: сборку переносимого пакета, его сериализацию и слияние принятого пакета с базой; `RelayEncryption` — за шифрование содержимого; `RelaySyncMetadataRepository` — за журнал применённых пакетов, обеспечивающий идемпотентность; `RelayQrCodec` — за кодирование пригласительных ссылок в QR-код (но не данных поездки, см. ниже).

Единица обмена — пакет `RelayPackage` (текущая версия формата — 2). Пакет содержит снимок поездки целиком: саму поездку, списки участников, дней, мест, расходов и их долей, а также журнал удалений `DeletionLog`. В заголовке пакета — его идентификатор `packageId`, идентификатор устройства-источника `deviceId`, время создания и поля поддержки дельта-обмена (`isDelta`, `baseCursor`, `changeCursor`), позволяющие передавать не весь снимок, а только изменения с известного момента. Для присоединения нового участника предусмотрен облегчённый пакет `InvitePackage` — без расходов и журнала удалений. Пакет сериализуется в JSON библиотекой `Gson` с отдельным адаптером для дат `LocalDate`. Существенно, что пакет и логика его слияния не зависят от способа доставки — это свойство используется ниже для объединения офлайн-обмена и онлайн-синхронизации.

В качестве транспорта выбран `Bluetooth Classic` с профилем `RFCOMM`, а не `Bluetooth Low Energy`. Обоснование дано в §1.2: пакет данных поездки весит сотни килобайт, тогда как BLE с его GATT-характеристиками рассчитан на короткие сообщения, а `RFCOMM` предоставляет потоковую передачу произвольного объёма [7, 8]. Обмен устроен асимметрично: отправитель выступает `RFCOMM`-клиентом и подключается к сервисному `UUID`, получатель — `RFCOMM`-сервером, слушающим этот `UUID`; обнаружение устройств идёт через системный механизм `Bluetooth-discovery` с приёмником широковещательных сообщений. Длина каждого передаваемого блока предваряется целочисленным префиксом, чтобы принимающая сторона знала, сколько байт читать.

Сессия обмена подчинена явному протоколу рукопожатия (рис. 6, вверху). Каждый кадр начинается сигнатурой `TRIL00_BT` и номером версии протокола, что отсекает посторонние подключения к тому же `UUID`. Обмен идёт в четыре шага: отправитель сообщает о намерении передать поездку (`INTENT` — идентификатор и название поездки, кто отправляет); получатель отвечает согласием или отказом (`CONSENT`); при согласии отправитель передаёт зашифрованный пакет (`PACKAGE`); получатель применяет его и возвращает подтверждение

(RESPONSE со статусом APPLIED, DUPLICATE или FAILED). Шаг подтверждения согласия отвечает требованию приватности (N5): данные поездки не передаются без явного действия принимающей стороны, а ожидание согласия ограничено таймаутом.

Содержимое пакета шифруется перед отправкой в эфир (рис. 6, внизу). Применён алгоритм AES-256 в режиме GCM (AES/GCM/NoPadding), обеспечивающий и конфиденциальность, и проверку целостности [6]. Ключ сессии выводится из общего секрета — идентификатора поездки `tripId`, известного обеим сторонам, — хешированием SHA-256, дающим 256-битный ключ. Перед шифрованием генерируется 12-байтный вектор инициализации (IV) на основе криптографически стойкого источника случайности; зашифрованный блок передаётся как конкатенация IV и шифртекста со встроенным 128-битным тегом аутентификации. При расшифровке несовпадение тега — следствие искажения хотя бы одного байта в эфире — приводит к отказу, что отвечает критерию безопасности канала из §1.4. Принятая модель угроз ограничена пассивным прослушиванием эфира в зоне действия Bluetooth; усиление вывода ключа (обмен эфемерными ключами или деривация с солью вместо хеша идентификатора) отнесено к перспективам развития.

Слияние принятого пакета с локальной базой выполняет `RelayRepository`. Для каждой сущности применяется правило Last-Write-Wins: если записи с таким идентификатором локально нет — она вставляется; если есть и у входящей версии метка `updatedAt` позже локальной — локальная замещается; иначе входящая отбрасывается. Удаления применяются по журналу `DeletionLog`: запись удаляется, только если время её удаления не раньше времени последнего изменения локальной версии, — это не даёт «воскресить» сущность устаревшим пакетом и, наоборот, отменить актуальное изменение устаревшим удалением. Идемпотентность обеспечивается на двух уровнях. Во-первых, правило Last-Write-Wins само по себе делает повторное применение того же пакета безвредным: метки времени совпадают, и ничего не меняется. Во-вторых, `RelaySyncMetadataRepository` хранит идентификаторы уже применённых пакетов, и повторно пришедший пакет распознаётся и подтверждается статусом DUPLICATE без повторного слияния (N4).

Поскольку и пакет `RelayPackage`, и его сборка, сериализация и слияние не зависят от транспорта, тот же доменный слой обслуживает и онлайн-синхронизацию. Репозиторий `OnlineSyncRepository` отправляет и принимает те же пакеты через HTTPS-вызовы собственного backend (`api/v1/sync/push` и `api/v1/sync/pull`) и сливает их той же функцией `RelayRepository`, с той же стратегией Last-Write-Wins и тем же журналом удалений. Смена транспорта — Bluetooth RFCOMM или REST поверх HTTPS — не затрагивает ни репозитории доменной логики, ни UI: меняется только класс, доставляющий пакет. Это и есть ключевое архитектурное решение работы, заявленное во введении как элемент новизны: единый доменный слой для офлайн-обмена и онлайн-синхронизации. Пошаговое описание протокола

обмена и онлайн-синхронизации вынесено в §3.3–3.4.

Близкий по характеру, но отдельный сценарий — взаимный геошаринг участников. Он реализован поверх Foreground Service (LocationSharingForeground) который при явном согласии участника периодически (с интервалом порядка 15–20 секунд) обновляет его координаты в локальной базе и на backend и отмечает участника как активного. Геошаринг не входит в пакет офлайн-обмена и работает только при наличии сети; ограничения по частоте обновлений отвечают требованию N3, а явное согласие — требованию N5.

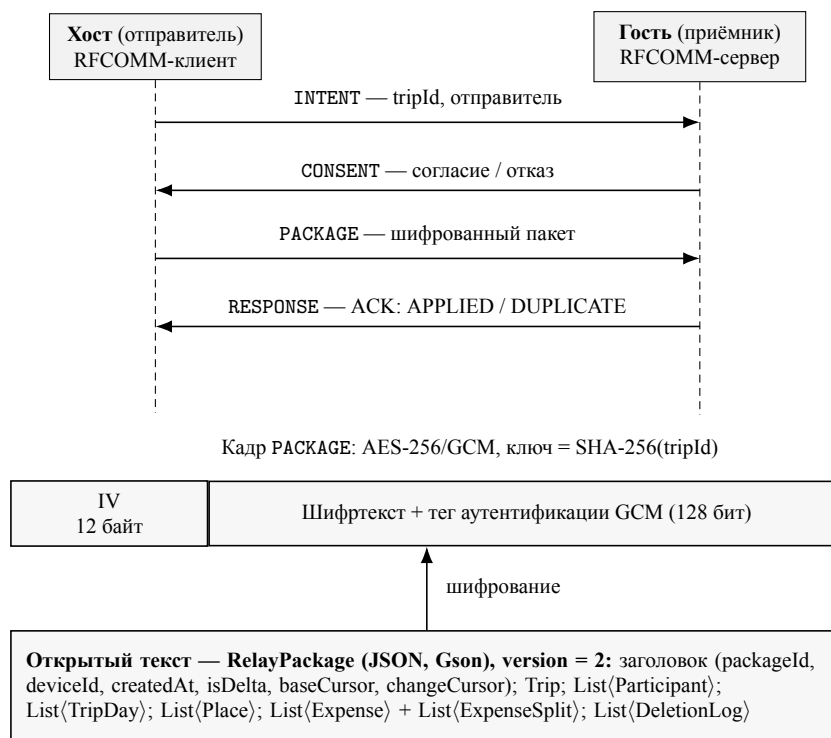


Рис. 6. Протокол сессии Triloo Relay (вверху) и структура шифрованного пакета поездки (внизу)

Полный сценарий офлайн-обмена как взаимодействие компонентов двух устройств показан на рис. 7: хост собирает и шифрует пакет, передаёт его после согласия гостя; гость расшифровывает пакет, сливает его в локальную базу по правилу Last-Write-Wins с учётом журнала удалений и возвращает подтверждение.

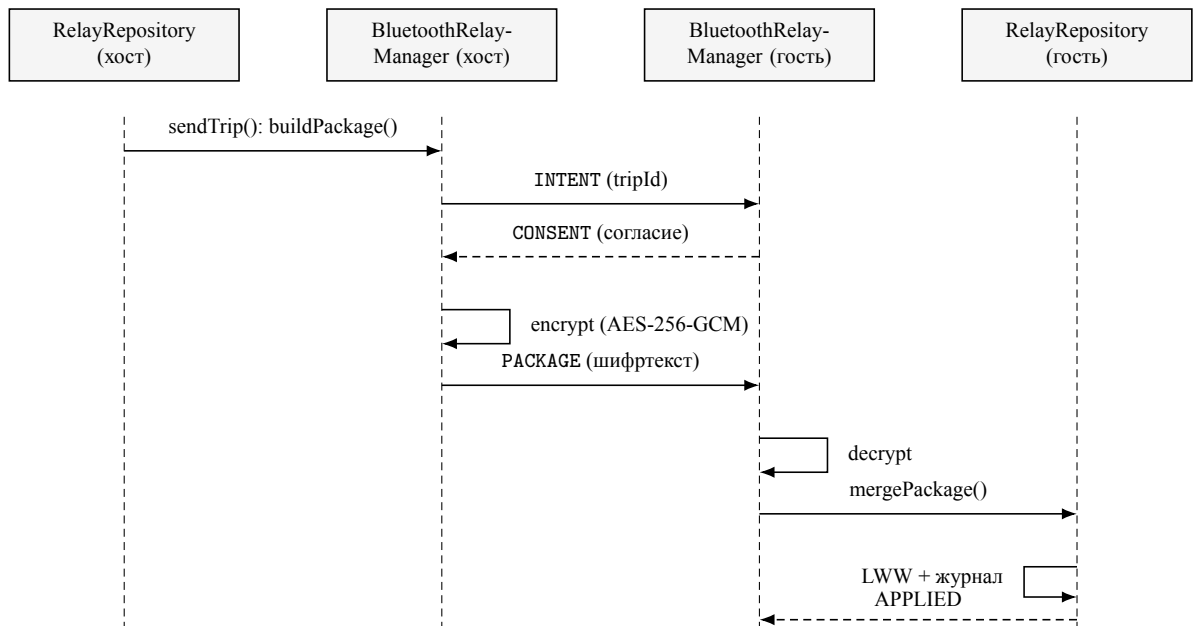


Рис. 7. UML-диаграмма последовательности: офлайн-обмен поездкой через Triloo Relay

2.5. Выводы по главе 2

В главе спроектирована архитектура приложения Triloo, отвечающая требованиям из §1.4. Клиент построен как многоуровневая offline-first-система на шаблоне MVVM с чистым разделением слоёв, где локальная база Room служит единственным источником истины для интерфейса, а сеть выступает опциональным каналом (задача 1). Спроектирована доменная модель поездки из восьми сущностей и её отображение на локальное хранилище Room с UUID-ключами, временными метками для слияния и журналом удалений; решения обоснованы требованиями офлайн-обмена и сохранности данных (задача 2). Интеграционный слой изолирует внешние сервисы за репозиториями и интерфейсами так, что их недоступность не нарушает работу локального ядра. Наконец, спроектирована подсистема офлайн-обмена Triloo Relay: формат переносимого пакета, протокол сессии поверх Bluetooth Classic с шифрованием AES-256-GCM и слияние по правилу Last-Write-Wins с журналом удалений; показано, что тот же доменный слой обслуживает и онлайн-синхронизацию (задача 8, элемент новизны). Спроектированная архитектура служит основой для главы 3, где описаны алгоритмы, реализующие отдельные подсистемы: эвристическая оптимизация маршрута, мультивалютный расчёт балансов и упрощение долгов, протокол обмена Triloo Relay и онлайн-синхронизация с backend.

ГЛАВА 3. РЕАЛИЗАЦИЯ КЛЮЧЕВЫХ ПОДСИСТЕМ TRILOO

Архитектура из главы 2 определяет, как приложение разбито на слои и как данные движутся между ними. Настоящая глава описывает реализацию алгоритмов четырёх подсистем: эвристической оптимизации порядка обхода точек (§3.1), мультивалютного расчёта балансов и упрощения долгов (§3.2), протокола офлайн-обмена Triloo Relay (§3.3) и онлайн-синхронизации с собственным backend (§3.4). Каждая подсистема рассматривается от постановки к коду: приводятся расчётные формулы, фрагменты реализации и схемы работы. Экспериментальная проверка перечисленных решений вынесена в главу 4.

3.1. Эвристическая оптимизация порядка обхода точек

День поездки — это набор мест с координатами, и порядок их обхода прямо влияет на суммарный путь. Задача упорядочивания точек так, чтобы маршрут был коротким, — это задача коммивояжёра в открытой постановке (без возврата в начало). Она NP-трудна [13]: точное решение перебором требует порядка $n!$ вариантов и неприменимо уже при десятке-полтора точек. Для прикладной задачи планирования дня этого и не требуется — достаточно быстрого приближённого решения, и в Triloo применена эвристика ближайшего соседа [4, 5], реализованная классом `NearestNeighborRouteOptimizer`.

Алгоритм стартует из фиксированной точки — ближайшей к текущему положению пользователя, а если положение неизвестно, из первого места списка — и на каждом шаге переходит к ближайшему ещё не посещённому месту, пока не обойдёт все (листинг 1). Его вычислительная сложность — $O(n^2)$: на каждом из n шагов перебираются оставшиеся точки.

Листинг 1 — алгоритм ближайшего соседа (`RouteOptimizer.kt`)

```
private fun nearestNeighbor(  
    places: List<Place>, startLocation: LatLng?  
) : List<Place> {  
    val remaining = places.toMutableList()  
    val result = mutableListOf<Place>()  
    var curLat = startLocation?.latitude ?: places.first().latitude  
    var curLon = startLocation?.longitude ?: places.first().longitude  
    while (remaining.isNotEmpty()) {  
        val nearest = remaining.minByOrNull { p ->  
            haversineDistance(curLat, curLon, p.latitude, p.longitude)  
        }!!  
        result.add(nearest)  
        remaining.remove(nearest)  
        curLat = nearest.latitude
```



```

        curLon = nearest.longitude
    }
    return result
}

```

Мерой близости служит геодезическое расстояние по формуле гаверсинусов (1) — длина дуги большого круга между двумя точками на сфере радиуса $R = 6\,371\,000$ м:

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\varphi_2 - \varphi_1}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\lambda_2 - \lambda_1}{2}}. \quad (1)$$

Выбор геодезического расстояния, а не реального дорожного, на этапе упорядочивания продиктован требованием offline-first (N2). Упорядочивание точек должно работать без сети, а формула (1) считается локально, мгновенно и не зависит от внешних сервисов. Реальные дорожные расстояния и время в пути подключаются на следующем этапе — при построении итогового маршрута для отображения (calculateRoute): дистанции и длительности отдельных переходов, а также полилиния трассы запрашиваются у маршрутных сервисов Yandex MapKit [9] и OpenRouteService [17], а при их недоступности приложение возвращается к оценке по формуле (1). Геодезическое расстояние монотонно связано с дорожным и служит дешёвым приближением, достаточным для выбора разумного порядка обхода.

Разделение «упорядочивание — построение маршрута» закреплено в интерфейсах (§2.3): RouteOptimizer отвечает за порядок точек, MapRouteProvider — за построение трассы у внешнего сервиса, NearbyPlacesProvider — за поиск мест рядом; все три связаны с реализациями в модуле RouteModule, поэтому маршрутный провайдер заменяется без правки вызывающего кода. Распределение мест по дням и подсказку режима передвижения берёт на себя отдельный компонент RoutePlanningAssistant: внутри каждого дня он применяет тот же ближайший сосед, а способ перемещения (пешком, велосипед, транспорт, автомобиль) выбирает эвристически по суммарной протяжённости дня. При наличии сети ассистент может запросить переупорядочивание у внешней модели, но эвристический результат всегда вычисляется первым и служит основой и резервом.

Принцип работы эвристики показан на рис. 8: из стартовой точки маршрут последовательно «притягивается» к ближайшему непосещённому месту. Полученный порядок короче исходного (в котором пользователь добавлял места произвольно), хотя и не обязан совпадать с оптимальным. Насколько именно сокращается длина и насколько результат отстоит от оптимума, измерено в §4.3.

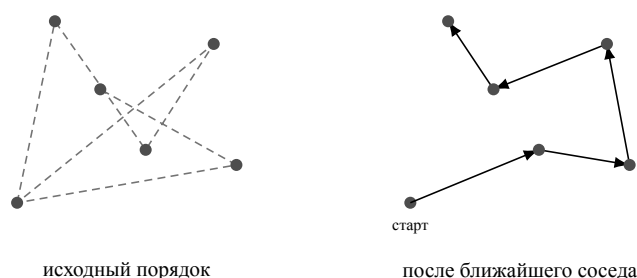


Рис. 8. Упорядочивание точек дня эвристикой ближайшего соседа (схематично)

3.2. Мультивалютный расчёт балансов и упрощение долгов

Финансовый модуль решает три связанные задачи (F6–F8): учесть расходы в разных валютах, посчитать, кто кому сколько должен, и свести долги к небольшому числу переводов. Все три опираются на единое приведение сумм к базовой валюте поездки.

Каждый расход хранит сумму в валюте операции (`amount`, `currency`) и её эквивалент в базовой валюте поездки (`amountInBaseCurrency`) вместе с применённым курсом (`exchangeRate`) и датой курса (`exchangeRateDate`). Курс предоставляет метод `getOrFetchCurrencyRate`: сначала он ищет курс на нужную дату в локальном кэше `CurrencyRate`, при отсутствии берёт последний известный; если кэшированный курс устарел (старше 24 часов), запрашивает свежий у внешнего сервиса (`ExchangeRate-API`) и сохраняет его в кэш. Когда сети нет, возвращается последний доступный курс из кэша, а пользователь получает предупреждение о его давности (N7). Перевод в базовую валюту снимает разнородность валют: дальнейший расчёт балансов идёт уже в одной валюте.

Способ деления расхода задаётся типом сплита. Реализованы три режима (табл. 5); режимы `PERCENTAGE` и `SHARES` предусмотрены моделью данных и отнесены к дальнейшему развитию. Доля каждого участника пересчитывается в базовую валюту тем же курсом, что и сам расход, что сохраняет согласованность сумм.

Таблица 5 — Реализованные режимы деления расхода

Режим	Как делится расход	Доля участника
<code>PAYER_ONLY</code>	личный расход, доли не создаются	не участвует в балансах
<code>EQUAL</code>	сумма поровну между участниками	a/N
<code>EXACT</code>	заданные суммы по участникам	s_u (из <code>splitAmounts</code>)

По расходам и долям считается чистый баланс каждого участника `calculateBalances`. Расходы со статусом «погашено» и личные расходы (`PAYER_ONLY`) из расчёта исключаются. Для остальных плательщик кредитует-ся на всю сумму, а каждый участник дебетуется на свою долю; все величины — в базовой валюте. Чистый баланс участника u выражается формулой (2):

$$b_u = \sum_{e: u \in \text{split}(e)} s_{e,u} - \sum_{e: \text{payer}(e)=u} a_e, \quad (2)$$

где $s_{e,u}$ — доля участника u в расходе e , a_e — сумма расхода e . Положительный b_u означает, что участник должен, отрицательный — что должны ему.

Имея чистые балансы, модуль сводит их к минимальному набору переводов жадным алгоритмом `simplifyDebts` (листинг 2). На каждом шаге выбирается участник с наибольшим долгом и участник с наибольшим встречным кредитом, между ними проводится перевод на меньшую из двух сумм, после чего остатки обновляются и закрытые участники выбывают. Денежные величины обрабатываются типом `BigDecimal` с округлением до копейки (`HALF_UP`), а остатки меньше порога 0,01 отбрасываются как незначимые.

Листинг 2 — упрощение долгов (`ExpenseRepository.kt`)

```
while (debtors.isNotEmpty() && creditors.isNotEmpty()) {
    val debtor = debtors.maxByOrNull { it.value } ?: break
    val creditor = creditors.minByOrNull { it.value } ?: break
    val amount = minOf(debtor.value, creditor.value.negate())
    if (amount > threshold) {
        balances.add(Balance(
            fromUserId = debtor.key, toUserId = creditor.key,
            amount = amount.setScale(2, RoundingMode.HALF_UP).toDouble(),
            currency = currency))
    }
    debtors[debtor.key] = debtor.value - amount
    creditors[creditor.key] = creditor.value + amount
}
```

Каждый перевод закрывает баланс хотя бы одного участника, поэтому их число не превышает $n - 1$, тогда как наивный попарный расчёт даёт до $n(n - 1)/2$ переводов. Алгоритм не гарантирует глобально минимального числа переводов (эта задача в общем случае также трудна), но на практике даёт близкий к минимуму результат за линейное по числу участников время. Работа алгоритма на примере из трёх участников показана на рис. 9: Alice оплачивает расход на 120 у.е., Bob — на 30 у.е., оба делятся поровну на троих; чистые балансы составляют -70 , $+20$ и $+50$, и долги сводятся к двум переводам в адрес Alice. Этот пример воспроизведён модульным тестом (§4.2).

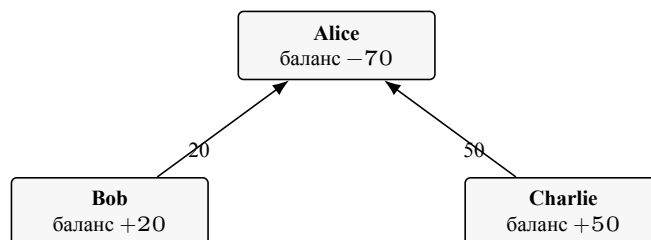


Рис. 9. Сведение чистых балансов к минимальному набору переводов

3.3. Протокол обмена Triloo Relay

Глава 2 описала Triloo Relay как подсистему, передающую пакет поездки между устройствами без сети и сливающую его с локальной базой приёмника (рис. 6, 7). Здесь разобрана реализация трёх её частей: транспорта, шифрования и слияния.

Транспортом служит Bluetooth Classic с профилем RFCOMM. Роли в сессии асимметричны: отправитель («хост») выступает RFCOMM-клиентом и подключается к сервисному UUID, а получатель («гость») — RFCOMM-сервером, слушающим этот UUID. Обмен идёт кадрами фиксированного формата: каждый кадр открывается сигнатурой TRIL00_BT и номером версии протокола, что отсекает посторонние подключения к тому же UUID. Длина кадра с пакетом предваряется целочисленным префиксом, чтобы приёмник знал, сколько байт читать. Сессия проходит четыре шага рукопожатия — INTENT, CONSENT, PACKAGE, RESPONSE (рис. 6), а в ответе передаётся один из статусов: APPLIED, DUPLICATE или FAILED.

Содержимое пакета шифруется перед отправкой алгоритмом AES-256 в режиме GCM (AES/GCM/NoPadding), который обеспечивает и конфиденциальность, и контроль целостности [6]. Ключ выводится хешированием SHA-256 от общего секрета — идентификатора поездки `tripId`, известного обеим сторонам после приглашения, поэтому отдельный обмен ключами не нужен. Перед каждым шифрованием генерируется случайный 12-байтный вектор инициализации; на выход подаётся конкатенация вектора и шифртекста со встроенным 128-битным тегом аутентификации (листинг 3). При расшифровке несовпадение тега — следствие искажения хотя бы одного байта или неверного ключа — приводит к отказу, что и составляет проверку целостности канала.

Листинг 3 — шифрование пакета AES-256-GCM (`RelayEncryption.kt`)

```
fun encrypt(plaintext: ByteArray, key: SecretKey): ByteArray {
    val iv = ByteArray(12).also { SecureRandom().nextBytes(it) }
    val cipher = Cipher.getInstance("AES/GCM/NoPadding")
    cipher.init(Cipher.ENCRYPT_MODE, key, GCMParameterSpec(128, iv))
    return iv + cipher.doFinal(plaintext)
}

fun decrypt(data: ByteArray, key: SecretKey): ByteArray {
    val iv = data.copyOfRange(0, 12)
    val cipher = Cipher.getInstance("AES/GCM/NoPadding")
    cipher.init(Cipher.DECRYPT_MODE, key, GCMParameterSpec(128, iv))
    return cipher.doFinal(data.copyOfRange(12, data.size))
}
```

Принятая модель угроз ограничена пассивным прослушиванием эфира в зоне действия Bluetooth. Ключ детерминированно выводится из `tripId` и не

является эфемерным, поэтому защита от активного перехвата с подменой при утечке `tripId` не обеспечивается; усиление вывода ключа (эфемерные ключи или деривация с солью) отнесено к перспективам развития.

Слияние принятого пакета выполняет `mergePackage` по правилу Last-Write-Wins, единому для всех сущностей (листинг 4): если записи с таким идентификатором локально нет, она вставляется; если есть и у входящей версии метка `updatedAt` позже локальной, локальная замещается; иначе входящая отбрасывается. Удаления применяются по журналу `DeletionLog` отдельно: запись удаляется, только если время её удаления не раньше времени последнего изменения локальной версии. Это правило не даёт устаревшему удалению отменить актуальную правку и, наоборот, не даёт устаревшей правке воскресить уже удалённую запись.

Листинг 4 — правило слияния Last-Write-Wins (`RelayRepository.kt`)

```
val local = tripDao.getTripById(trip.id)
when {
    local == null -> { tripDao.insertTrip(trip); inserted++ }
    trip.updatedAt > local.updatedAt -> { tripDao.updateTrip(trip); u
    else -> Unit
}
```

Идемпотентность приёма обеспечена на двух уровнях. Само правило Last-Write-Wins делает повторное применение того же пакета безвредным: метки времени совпадают, и слияние не меняет ни одной записи (это подтверждено тестом в §4.4). Дополнительно `RelaySyncMetadataRepository` хранит идентификаторы уже применённых пакетов (последние 50), и повторно пришедший пакет распознаётся и подтверждается статусом `DUPLICATE` без повторного слияния. Тот же механизм поддерживает дельта-обмен: метод `buildPackage` умеет собирать не весь снимок, а только сущности, изменившиеся после заданного курсора (`updatedAt` или `deletedAt` больше `sinceCursor`), пометая пакет флагом `isDelta` и сохраняя границы `baseCursor` и `changeCursor`.

3.4. Онлайн-синхронизация с собственным backend

Главное архитектурное решение работы, заявленное во введении как элемент новизны, — единый доменный слой для офлайн-обмена и онлайн-синхронизации. Репозиторий `OnlineSyncRepository` не повторяет логику `Relay`, а переиспользует её целиком: сборку пакета (`buildPackage`), его сериализацию и разбор (`encodePackage`, `decodePackage`) и слияние (`mergePackage`). По сети передаётся тот же `RelayPackage` с той же стратегией Last-Write-Wins и тем же журналом удалений; меняется лишь транспорт — вместо кадров Bluetooth используются HTTPS-вызовы.

Отправка изменений (`syncTripNow`) собирает пакет поездки, кодирует его и отправляет методом `POST api/v1/sync/push` с токеном авторизации; в ответе

сервер сообщает, какие поездки приняты, а какие отклонены. Приём изменений (`pullRemoteChanges`) запрашивает `GET api/v1/sync/pull` с курсором `since`, получает накопленные на сервере снимки и сливает каждый тем же методом `mergePackage`. Курсор делает синхронизацию инкрементальной: сервер отдаёт только снимки, обновлённые после последнего известного клиенту момента.

В отличие от Bluetooth-обмена, где доверие держится на физической близости устройств, онлайн-канал добавляет ролевой контроль на стороне сервера. Запросы авторизуются токеном (`ServerSessionRepository`), а сервер проверяет роль участника: владелец и администратор могут изменять структуру поездки, участник с ролью `MEMBER` — только свои расходы, передача владения через обычную синхронизацию запрещена; на выдаче сервер фильтрует снимки по членству пользователя в поездке. Так ролевая модель `OWNER/ADMIN/MEMBER` из доменной модели получает реальное разграничение прав, а не только хранение роли.

Серверная часть (Node.js, Express, SQLite) реализует аутентификацию, энд-поинты `push` и `pull` с описанным ролевым контролем и развёрнута под управлением процесс-менеджера. На текущем этапе онлайн-синхронизация доведена до рабочего минимально жизнеспособного состояния: основное слияние выполняется на клиенте по правилу `Last-Write-Wins`, а сервер проверяет права и свежесть снимка. Промышленная закалка — интеграционные тесты, обязательная верификация почты, веб-панель администрирования, серверное слияние на уровне отдельных полей — отнесена к дальнейшему развитию (см. заключение).

3.5. Выводы по главе 3

В главе описана реализация четырёх ключевых подсистем приложения. Оптимизация порядка обхода точек построена на эвристике ближайшего соседа по геодезическим расстояниям с подключением реальных дорожных дистанций на этапе отрисовки маршрута (задача 5). Финансовый модуль приводит расходы к базовой валюте по кэшируемым курсам, считает чистые балансы участников и сводит долги к набору не более чем $n - 1$ переводов жадным алгоритмом (задача 6). Подсистема `Triloo Relay` реализует обмен пакетом поездки по Bluetooth Classic с шифрованием `AES-256-GCM` и слиянием по правилу `Last-Write-Wins` с журналом удалений и двухуровневой идемпотентностью (задача 8). Онлайн-синхронизация переиспользует тот же доменный слой поверх `HTTPS`, добавляя серверный ролевой контроль, чем подтверждается заявленный элемент новизны — независимость доменной логики от транспорта (задача 10). Корректность и качество этих решений проверяются в главе 4.

ГЛАВА 4. ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА ПРИЛОЖЕНИЯ TRILOO

Глава проверяет, выполнены ли критерии качества, сформулированные в §1.4. Каждый критерий проверяется тем методом, который ему соответствует: доменная логика — модульными тестами на JVM, качество маршрута и стойкость канала — отдельными замерочными тестами, а сценарные требования — прогоном сквозного сценария на устройстве. Методика и среда описаны в §4.1, далее по разделам идут покрытие функциональных требований (§4.2), качество маршрута (§4.3), корректность слияния пакетов Relay (§4.4), безопасность канала (§4.5) и поведение без сети (§4.6).

4.1. Методика и среда проверки

Проверка опирается на пять критериев качества из §1.4: покрытие функциональных требований F1–F15, корректность слияния пакетов при офлайн-обмене, качество маршрута на выходе оптимизатора, поведение в офлайн-режиме и безопасность канала Triloo Relay.

Доменная логика проверяется модульными тестами на JVM (JUnit 4, Robolectric, база Room в памяти) — без эмулятора и без сети, что отвечает требованию тестируемости N6. Так проверяются расчёт балансов и упрощение долгов, слияние пакетов Relay и шифрование канала. Качество маршрута измеряется отдельным тестом, сравнивающим длину маршрута эвристики с оптимумом. Сценарные требования, требующие интерфейса и внешних сервисов, проверяются на устройстве прогоном сквозного сценария «Сеул, 3 дня, 2 участника, расходы в нескольких валютах». Поведение без сети оценивается качественно — наблюдением за работой ключевых экранов при отключённом интернете. Все модульные тесты запускаются командой `./gradlew testDebugUnitTest`.

4.2. Покрытие функциональных требований

Все требования F1–F15 имеют рабочую реализацию и воспроизводятся на сквозном сценарии «Сеул». Способ проверки и результат по каждому требованию сведены в табл. 6. Часть требований (F3, F4, F10, F14) по своей природе зависит от внешних сервисов и выполняется при наличии сети, что согласуется с принципом *offline-first* (N2): локальное ядро остаётся работоспособным и без них. Основные экраны работающего приложения приведены на рис. 10.

Таблица 6 — Покрытие функциональных требований на сценарии «Сеул»

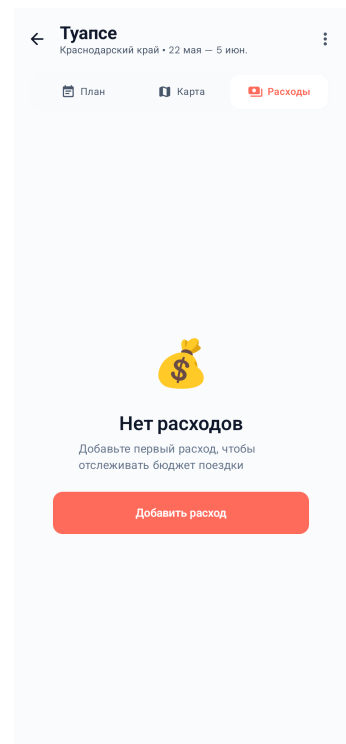
Код	Способ проверки	Результат
F1	Создание поездки «Сеул» с датами и базовой валютой, редактирование и удаление	реализовано
F2	Авторазбиение поездки на 3 дня, добавление мест в день, перестановка, отметка «посещено»	реализовано
F3	Поиск места с автодополнением, извлечение координат и атрибутов	реализовано (при сети)
F4	Маркеры на карте, полилиния маршрута, расстояние и время в пути	реализовано (при сети)
F5	Оптимизация порядка точек, замер качества	реализовано; тест (§4.3)
F6	Добавление, редактирование, удаление расходов, сводные показатели	реализовано
F7	Конвертация в базовую валюту по кэшу курсов, предупреждение о давности курса	реализовано
F8	Режимы сплита (PAYER_ONLY, EQUAL, EXACT), балансы и минимальный набор переводов	реализовано; тест
F9	Приглашение по 6-символьному коду и QR, роли OWNER/ADMIN/MEMBER	реализовано
F10	Геошаринг через Foreground Service по явному согласию	реализовано (при сети)
F11	Обмен пакетом по Bluetooth, шифрование, слияние с журналом удалений	реализовано; тесты (§4.4, §4.5)
F12	Синхронизация push/pull с backend, ролевой контроль на сервере	реализовано (MVP, §3.4)
F13	On-device OCR чека, извлечение суммы, валюты и даты	реализовано; тест парсера
F14	Ранжирование жилья по эвристикам поверх Geoapify	реализовано (при сети)
F15	Тема, локализация (ru/en), уведомления, экспорт данных	реализовано



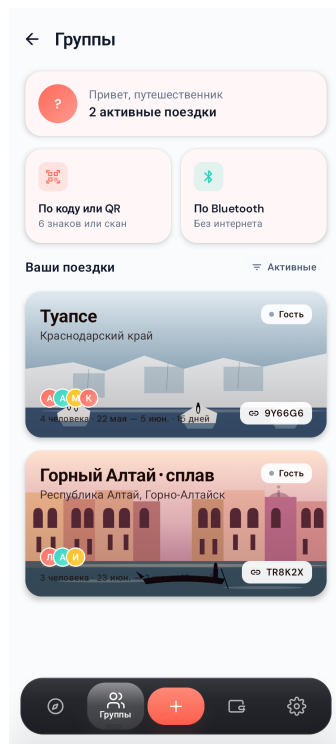
a



б



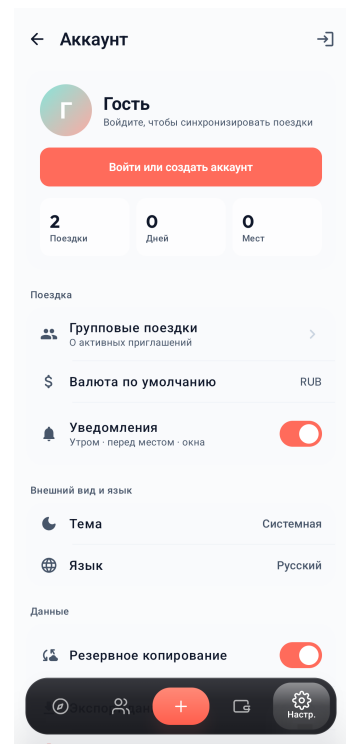
в



г



д



е

Рис. 10. Основные экраны приложения Triloo: *a* — список поездок; *б* — карта поездки с участниками; *в* — учёт расходов; *г* — группы с инвайт-кодами и ролями; *д* — офлайн-обмен Triloo Relay; *е* — настройки

4.3. Качество маршрута на выходе оптимизатора

Качество эвристики ближайшего соседа измерено замерочным тестом `RouteQualityMeasurementTest`. Для каждого числа точек $N \in \{5, 7, 10, 12, 15\}$ генерировалось по 20 случайных наборов координат в пределах ограничивающего прямоугольника Сеула (фиксированный сид обеспечивает воспроизводимость). Для каждого набора измерялись три длины по формуле гаверсинусов: маршрут в исходном порядке (порядок добавления точек, здесь — случайный), маршрут после ближайшего соседа (выход реального оптимизатора) и оптимальный открытый маршрут, вычисленный методом Хелда–Карпа. По длинам считались выигрыш над исходным порядком и отклонение от оптимума; усреднённые по 20 наборам результаты приведены в табл. 7.

Таблица 7 — Качество маршрута ближайшего соседа (среднее по 20 наборам)

N	Исходный, км	NN, км	Оптимум, км	Выигрыш, %	Откл. от опт., %
5	48,6	33,8	29,6	28,8	15,3
7	72,2	44,2	39,3	37,4	12,9
10	96,7	52,5	47,1	44,5	11,5
12	117,7	61,6	52,1	46,4	18,2
15	161,3	69,7	59,2	56,2	17,9

Эвристика сокращает длину маршрута относительно исходного порядка на 28,8–56,2%, и выигрыш растёт с числом точек: чем больше точек добавлено в произвольном порядке, тем больше выигрывает упорядочивание. Отклонение от оптимума остаётся в пределах 11,5–18,2%, что укладывается в типичные для ближайшего соседа значения [4, 5]; на равномерно случайных входах среднее отклонение эвристики обычно ниже её худшего случая. Большие абсолютные длины объясняются тем, что точки разбросаны по всей площади города; для реального дня поездки с близкими точками длины меньше, но соотношения сохраняются. Замечание: в самом приложении показатель экономии обрезается снизу нулём (пользователю не показывается отрицательная экономия), тогда как в эксперименте приводятся истинные значения.

4.4. Корректность слияния пакетов Triloo Relay

Слияние проверяется модульными тестами над репозиторием `RelayRepository` с базой `Room` в памяти. Проверяются три свойства из критерия качества §1.4 — отсутствие дублирования при повторном приёме, невозвращение удалённых записей и приоритет более поздней правки — плюс корректность дельта-выборки. Сценарии и результаты сведены в табл. 8; все проверки пройдены.

Таблица 8 — Проверки слияния пакетов Triloo Relay

Сценарий	Ожидаемое поведение	Результат
Входящее удаление новее локальной записи (удаление $t = 5000$, запись $t = 1000$)	запись удаляется, новая запись из пакета вставляется	пройдено
Локальная версия новее входящей (локальная $t = 10000$, входящая $t = 4000$)	локальная версия сохраняется, входящая отбрасывается	пройдено
Повторное применение того же пакета	0 вставок, 0 обновлений, 0 удалений; число записей неизменно	пройдено
Сборка дельты от курсора ($sinceCursor = 5000$)	в пакет попадают только сущности, изменённые после курсора	пройдено

Эти проверки покрывают журнал удалений (удаление с меткой позже локальной правки убирает запись и не даёт ей вернуться при следующей синхронизации), правило Last-Write-Wins по метке `updatedAt`, идемпотентность (повторный приём идентичного пакета не меняет ни одной записи, так как метки времени совпадают) и корректность дельта-выборки. Критерий корректности слияния выполнен.

4.5. Безопасность канала Triloo Relay

Стойкость шифрования канала проверяется тестом `RelayEncryptionTest` над модулем `RelayEncryption` (AES-256-GCM). Проверено четыре свойства. Прямое преобразование обратимо: расшифровка зашифрованного текста тем же секретом возвращает исходные данные. Расшифровка с другим секретом (другой `tripId`) завершается отказом с ошибкой проверки тега `AEADBadTagException`. Искажение хотя бы одного байта шифртекста также приводит к отказу по тегу — это и есть контроль целостности. Наконец, повторное шифрование одного и того же текста даёт разный результат, поскольку перед каждым шифрованием генерируется случайный вектор инициализации. Все четыре проверки пройдены.

Из этого следует критерий безопасности §1.4: пакет, перехваченный в эфире, не читается без знания секрета сессии, а любая модификация пакета отвергается приёмником на этапе проверки тега GCM [6]. Принятая модель угроз (пассивное прослушивание) и её ограничения обсуждались в §3.3.

4.6. Поведение в офлайн-режиме

Поведение без сети оценивалось качественно — наблюдением за ключевыми сценариями при отключённом интернете. Просмотр поездки, добавление и редактирование расхода, расчёт балансов и обмен пакетом по Bluetooth выполня-

ются без сети, потому что источником истины служит локальная база Room, а не ответы сервера (§2.1, N2). Сеть требуется лишь тому, что объективно от неё зависит: тайлам карты, автодополнению мест, маршрутным API, курсам валют и онлайн-синхронизации; их недоступность отключает соответствующую функцию, но не затрагивает локальное ядро. Нестабильное Bluetooth-соединение восстанавливается в пределах таймаутов, а повторный приём одного и того же пакета не дублирует данные за счёт идемпотентности слияния (§4.4). Ключевые сценарии офлайн выполнимы и не приводят к отказу приложения, чем критерий поведения в офлайн-режиме выполнен.

4.7. Выводы по главе 4

Экспериментальная проверка подтвердила все пять критериев качества из §1.4. Требования F1–F15 имеют рабочую реализацию и воспроизводятся на сквозном сценарии (§4.2). Слияние пакетов Triloo Relay корректно и идемпотентно: подтверждены отсутствие дублирования, невозвращение удалённых записей и приоритет поздней правки (§4.4). Эвристика ближайшего соседа сокращает длину маршрута на 28,8–56,2% относительно исходного порядка при отклонении от оптимума 11,5–18,2% (§4.3). Канал Triloo Relay устойчив к пассивному прослушиванию и к искажению пакета (§4.5). Ключевые сценарии работают без сети (§4.6). Отмеченные ограничения — прежде всего состояние онлайн-синхронизации на уровне рабочего MVP — вынесены в перспективы развития (заключение).

Предварительный список использованных источников (рабочий)

Примечание. Раздел — рабочий список источников, на которые опираются главы 1 и 2. Финальный «СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ» формируется после завершения глав 2–4 и оформляется в соответствии с ГОСТ 7.1 и ГОСТ Р 7.0.5. Нумерация в этом разделе соответствует порядку упоминания ссылок в тексте глав 1 и 2.

1. Sauble D. Offline First Web Development. — Birmingham: Packt Publishing, 2015. — 240 с.
2. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol: O'Reilly Media, 2017. — 616 с.
3. Russell A. Building Progressive Web Apps: Bringing the Power of Native to the Browser. — Sebastopol: O'Reilly Media, 2017. — 320 с.
4. Rosenkrantz D. J., Stearns R. E., Lewis P. M. An Analysis of Several Heuristics for the Traveling Salesman Problem // SIAM Journal on Computing. — 1977. — Vol. 6, No. 3. — pp. 563–581.
5. Lawler E. L., Lenstra J. K., Rinnooy Kan A. H. G., Shmoys D. B. The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization. — Chichester: John Wiley & Sons, 1985. — 476 с.
6. Dworkin M. NIST Special Publication 800-38D. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. — Gaithersburg: National Institute of Standards and Technology, 2007. — 39 с.
7. Bluetooth Core Specification 5.4. — Kirkland: Bluetooth Special Interest Group, 2023. — URL: <https://www.bluetooth.com/specifications/specs/core-specification-5-4/> (дата обращения: 15.05.2026).
8. RFCOMM with TS 07.10. Serial Port Emulation: Bluetooth Specification. — Kirkland: Bluetooth Special Interest Group, 2003. — 31 с.
9. Yandex MapKit SDK for Android. Reference and Developer Guides // Yandex Maps API. — URL: <https://yandex.com/dev/maps/mapkit/doc/android-ref/> (дата обращения: 11.05.2026).
10. Guide to App Architecture // Android Developers. — Mountain View: Google, 2024. — URL: <https://developer.android.com/topic/architecture> (дата обращения: 10.05.2026).

11. Jetpack Compose. Developer Documentation // Android Developers. — Mountain View: Google, 2024. — URL: <https://developer.android.com/jetpack/com> (дата обращения: 10.05.2026).
12. Room Persistence Library. Developer Documentation // Android Developers. — Mountain View: Google, 2024. — URL: <https://developer.android.com/training/dstorage/room> (дата обращения: 10.05.2026).
13. Garey M. R., Johnson D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. — San Francisco: W. H. Freeman, 1979. — 338 с.
14. Shapiro M., Preguiça N., Baquero C., Zawirski M. Conflict-Free Replicated Data Types // INRIA Research Report. — Rocquencourt: INRIA, 2011. — No. 7687. — 50 с.
15. ML Kit Text Recognition v2 // Google Developers. — URL: <https://developers.google.com/ml-kit/vision/text-recognition/v2> (дата обращения: 12.05.2026).
16. Geoapify Places API Documentation // Geoapify GmbH. — URL: <https://apidocs.geoapify.com/docs/places/> (дата обращения: 11.05.2026).
17. OpenRouteService API Documentation // HeiGIT gGmbH. — URL: <https://openrouteservice.org/dev/> (дата обращения: 11.05.2026).
18. Wanderlog Travel Planner // Wanderlog Inc. — URL: <https://wanderlog.com/> (дата обращения: 08.05.2026).
19. Roadtrippers Travel Planner // Roadtrippers Inc. — URL: <https://roadtrippers.com/> (дата обращения: 08.05.2026).
20. Sygic Travel // Sygic a.s. — URL: <https://travel.sygic.com/> (дата обращения: 08.05.2026).
21. TripIt — Travel Planner with Flight Tracker // Concur Technologies. — URL: <https://www.tripit.com/> (дата обращения: 08.05.2026).
22. Notion — The All-in-One Workspace // Notion Labs Inc. — URL: <https://www.notion.so/> (дата обращения: 08.05.2026).
23. Obsidian — Sharpen Your Thinking // Dynalist Inc. — URL: <https://obsidian.md/> (дата обращения: 08.05.2026).
24. Polarsteps — Travel Tracker App // Polarsteps B.V. — URL: <https://www.polarsteps.com/> (дата обращения: 09.05.2026).
25. Visit a City — Travel Itinerary Planner // Visit a City Inc. — URL: <https://www.visitacity.com/> (дата обращения: 09.05.2026).

26. Travefy — Itinerary Builder for Travel Professionals // Travefy LLC. — URL: <https://travefy.com/> (дата обращения: 09.05.2026).
27. Splitwise. Split Bills and Expenses // Splitwise Inc. — URL: <https://www.splitwise.com/> (дата обращения: 08.05.2026).
28. Tricount: Friends' Shared Expenses // Bunq B.V. — URL: <https://www.tricount.com/> (дата обращения: 08.05.2026).
29. Settle Up — Group Expenses // Settle Up s.r.o. — URL: <https://www.settleup.io/> (дата обращения: 08.05.2026).
30. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017. — 32 с.
31. ГОСТ Р 7.0.99-2018. Система стандартов по информации, библиотечному и издательскому делу. Реферат и аннотация. Общие требования. — М.: Стандартинформ, 2018. — 12 с.
32. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» (в редакции от 24.07.2023) // Российская газета. — 2006. — 29 июля. — № 165.
33. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Boston: Prentice Hall, 2017. — 432 с.
34. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. — Reading: Addison-Wesley, 1994. — 395 с.
35. Dependency Injection with Hilt // Android Developers. — Mountain View: Google, 2024. — URL: <https://developer.android.com/training/dependency-injection/hilt-android> (дата обращения: 10.05.2026).
36. Coroutines and Flow. Kotlin Documentation // JetBrains. — URL: <https://kotlinlang.org/docs/coroutines-overview.html> (дата обращения: 10.05.2026).
37. Retrofit. A Type-safe HTTP Client for Android and Java // Square, Inc. — URL: <https://square.github.io/retrofit/> (дата обращения: 10.05.2026).